Slovak University of Technology in Bratislava
Faculty of Informatics and Information Technologies

FIIT-XXXX-XXXXX

Bc. Richard Čerňanský

Deep Learning Methods for Perception and Prediction in Autonomous Systems

Master's Thesis

Thesis Supervisor: Ing. Matej Halinkovič, PhD.

May 20xx

Slovak University of Technology in Bratislava
Faculty of Informatics and Information Technologies

FIIT-XXXX-XXXXX

Bc. Richard Čerňanský

Deep Learning Methods for Perception and Prediction in Autonomous Systems

Master's Thesis

Study Programme:	Intelligent Software Systems
Study Field:	Computer Science
Training Workplace:	Bratislava, Slovakia
Thesis Supervisor:	Ing. Matej Halinkovič, PhD.

May 20xx

Čestne vyhlasujem, že som túto prácu vypracoval samostatne, na základe konzultácií a s použitím uvedenej literatúry.

I declare in my honor that I have prepared this work independently, on the basis of consultations and using the mentioned literature.

In Bratislava, May 20xx

Bc. Richard Čerňanský

Annotation

Slovak University of Technology in Bratislava
Faculty of Informatics and Information Technologies
Study Programme: Intelligent Software Systems
Study Field: Computer Science

Author: Bc. Richard Čerňanský
Master's Thesis: Deep Learning Methods for Perception and Prediction in Autonomous Systems
Thesis Supervisor: Ing. Matej Halinkovič, PhD.
May 20xx

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Keywords: 3-5 keywords

Anotácia

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií
Študijný program: Inteligentné softvérové systémy
Študijný odbor: Informatika

Autor: Bc. Richard Čerňanský
Diplomová práca: Deep Learning Methods for Perception and Prediction in Autonomous Systems
Vedúci diplomového projektu: Ing. Matej Halinkovič, PhD.
Máj 20xx

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Oblasť problematiky: 3-5 kľúčových slov

BP2, DP3: sem vložte finalne a podpisane zadanie: PDF z AISu

BP1, DP1, DP2: sem vložte navrh zadania: PDF z YonBanu

Acknowledgements / Poďakovanie

The acknowledgments and the people to thank go here, don't forget to include your project advisor. . .

I also thank my family and friends who supported me during the writing of this thesis.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Formulation for Trajectory Prediction . . . 2
  - 1.2.1 Data Fusion . . . 3
  - 1.2.2 Dynamic Environments . . . 3
  - 1.2.3 Interaction Modeling . . . 3
  - 1.2.4 Real-World Constraints . . . 3
- 1.3 Thesis Outline . . . 4

2 Trajectory Prediction: Background and State of the Art **5**
- 2.1 Taxonomy of Trajectory Prediction Methods . . . 5
  - 2.1.1 Physics-Based Methods . . . 5
  - 2.1.2 Learning-based methods . . . 6
    - 2.1.2.1 Classical Machine Learning Methods . . . 6
    - 2.1.2.2 Reinforcement Learning Methods . . . 6
    - 2.1.2.3 Deep Learning Methods . . . 7
- 2.2 Prediction Paradigms . . . 8
  - 2.2.1 Modular (Detect-Track-Predict) . . . 8
  - 2.2.2 Joint Learning (End-to-End Perception & Prediction) . . . 9
- 2.3 Datasets . . . 9
- 2.4 Evaluation Metrics . . . 11
  - 2.4.1 Detection Metrics . . . 11
  - 2.4.2 Tracking Metrics . . . 12
  - 2.4.3 Trajectory Prediction Metrics . . . 14

3 Related Work **17**
- 3.1 Query-Based Detection and Tracking . . . 17
- 3.2 LiDAR Backbones and Heatmap Query Initialisation . . . 18
- 3.3 Attention-Based Camera-LiDAR Fusion . . . 18

3.4 Trajectory Decoding from Agent Features . . . 19
3.5 Positioning of This Work . . . 19
3.6 Summary and Starting Points . . . 20

4 Objectives and Methodology **23**
4.1 Objectives . . . 23
4.2 Methodology . . . 24

5 Design of the Proposed Model **25**
5.1 Requirements . . . 25
5.2 Overview . . . 26
5.3 The ViP3D Baseline . . . 26
5.3.1 Agent Queries and Track State . . . 27
5.3.2 Camera Feature Extraction . . . 28
5.3.3 Six-Layer Decoder with Projective Cross-Attention . . . 28
5.3.4 Prediction Branches . . . 28
5.3.5 Tracking . . . 29
5.3.6 Trajectory Prediction . . . 31
5.4 LiDAR Branch . . . 32
5.5 Query Initialisation from the LiDAR Heatmap . . . 33
5.5.1 Class-Wise Heatmap Head . . . 33
5.5.2 Selecting Peaks and Building Queries . . . 34
5.5.3 Integration with the Tracker . . . 36
5.6 Two-Layer Fusion Decoder . . . 37
5.6.1 Layer 0: Reading the BEV Map . . . 37
5.6.2 Layer 1: Spatially Modulated Camera Attention . . . 37
5.6.3 Reference Refinement . . . 38
5.6.4 Comparison with the Decoder of ViP3D . . . 39
5.7 Camera Visibility for Partial Camera Rigs . . . 40
5.8 Training . . . 40
5.8.1 Loss . . . 40
5.8.2 Two-Stage Schedule . . . 41
5.9 Complexity Analysis . . . 41
5.10 Summary . . . 44

6 Implementation **45**

7 Experiments and Evaluation **47**
7.1 Setup . . . 47
7.2 Case Study - Experiment 01 . . . 48

7.3 Case Study - Experiment 02 48
7.4 Case Study - Experiment 03 48
7.5 Summary 48

8 Conclusion **49**
8.1 Summary 49
8.2 Future Work 49

Resumé **49**

List of Abbreviations **51**

List of Figures **54**

List of Tables **55**

References **59**

A Description of Digital Submission **B-1**

B Technical Documentation **B-3**
B.1 Installation Manual B-3
B.2 User Manual / Používateľská príručka B-3
B.3 Developer Manual B-3

C Work Schedule **B-1**
C.1 Work Schedule for BP1/DP1 . . . . . B-1
C.2 Work Schedule for BP2/DP2 . . . . . B-2
C.3 Work Schedule for DP3 . . . . . B-2

Chapter 1

Introduction

In recent years, deep learning and computer vision methods have made significant advancements in the analysis of dynamic scenes and their applications in autonomous systems. The development of fields such as autonomous vehicles and robotics depends on solving tasks like object detection, object tracking, segmentation, and trajectory prediction. Specifically, autonomous vehicles (AVs) revolutionize transportantion by saving drivers' time, improving safety and possibly mitigating congestion on the roads. Current autonomous driving systems consist of multiple sequential tasks such as perception, tracking, prediction, planning and control [1]. Perception task is responsible for detecting and understanding the objects within the environment including their categorization, location and size estimation. The tracking objective uses the detections from perception to find the relationships between objects across the frames in scene. The prediction modules then process this tracking information in order to forecast future positions. The predicted positions enter the planning stage that generates the most feasible and optimal path in terms of safety, energy consuption and following traffic rules. Last but not least, controlling task then prepares actuator commands which execute the planned motion.

1.1 Motivation

Real traffic is highly dynamic and unpredictable: other drivers may behave erratically, and sensors provide noisy or incomplete data. Diverse maneuvers, complex interactions, and inherent errors in sensory information make prediction challenging, yet solving these challenges is essential for having trustworthy AVs. In addition, autonomous systems operate under strict constraints on latency and reliability [2]. Predictions must be computed in real time on embedded hardware and remain accurate over long hori-

zons to be useful for driving in real-life scenarios. Industry recognizes the need for fast execution, uncertainty handling, and generalization in prediction models to ensure robust operation in varied conditions [3, 4, 5]. Reliable trajectory prediction is therefore critical for safe autonomous driving and motivates intensive research into methods that can forecast the multi-agent future with practical efficiency. For these reasons, this thesis focuses on trajectory prediction, exploring how current state-of-the-art models could be modified or extended to achieve better performance and efficiency.

1.2 Problem Formulation for Trajectory Prediction

Trajectory prediction is the task of forecasting the future motion of the traffic participants that surround the ego vehicle. Its input is not a trajectory in the purely geometric sense, but rather the past states of these participants as they are captured in the environmental information collected by sensors, such as camera images, LiDAR point clouds, radar returns or HD maps providing lane geometry and traffic-rule priors. However, in this thesis only camera and LiDAR is employed. A model that relies on a single source is unimodal, whereas a model that exploits several of them at once is considered multi-modal. In the latter case the measurements are heterogeneous in dimensionality, sampling rate and coordinate frame, so the features extracted from the individual sources have to be fused into a unified representation of the scene before any forecasting takes place [6]. The output is the forecasted trajectory, that is, a sequence of future states of the observed agents over a fixed prediction horizon.

Formally, trajectory prediction can be stated as the problem of constructing a mapping Φ from the observed evidence $\mathcal{I}$ about the scene to the future states $\mathcal{O}$ of the traffic participants, from the current time up to a prediction horizon T [7]:

$$\begin{aligned} &\Phi : \mathcal{I} \rightarrow \mathcal{O}, \\ &\mathcal{O} = \left\{ S^0, S^1, \ldots, S^T \right\}, \end{aligned} \tag{1.1}$$

where S^0 is the current state and S^1 to S^T are the states at the individual future time steps, each of them collecting the states of all N observed participants. Depending on the available modalities and on the downstream task, a single state can take different forms, for example

$$S^t = \left\{ x^t, y^t \right\} \quad \text{or} \quad S^t = \left\{ x^t, y^t, l^t, w^t, \theta^t \right\}, \tag{1.2}$$

where x^t and y^t are the coordinates of the centre point of the object, l^t and w^t its length and width, and θ^t its orientation [7]. In trajectory prediction the state is usually reduced to the planar position, so that $\mathcal{O}$ corresponds to the set of forecasted trajectories of the

participants in the scene. Since the future of a traffic scene is inherently uncertain, the mapping in (1.1) is in practice often extended to output several plausible trajectories together with their confidences, allowing the planning stage to reason over the alternatives instead of committing to a single hypothesis [2]. The following subsections summarise the main challenges that this formulation raises.

1.2.1 Data Fusion

The information describing the scene comes from sources that are complementary but incompatible. Cameras provide dense semantic detail without depth and degrade at night, in glare or in poor lighting conditions, LiDAR provides accurate geometry but suffers in rain etc. These sources can be combined at the level of raw measurements (early), of learned features (intermediate) or of the individual predictions (late). Since every added modality costs latency and introduces noise the network is prone to overloading with parameters or can overspecialize on the most informative source and lose accuracy once the weather or illumination degrades it [6].

1.2.2 Dynamic Environments

Traffic is nonstationary, as density, weather, illumination and road layout change, so that the conditions met during deployment need not resemble those seen during development. The future is moreover not unique, since the same observed history is usually compatible with several maneuvers, and this aleatoric uncertainty originated in the erratic decisions of other drivers does not diminish with additional data. Therefore the task requires a set of socially compliant trajectories produced in time for the planning stage [6].

1.2.3 Interaction Modeling

The motions of traffic participants are not independent and traffic signals constrain all of them, so that the joint future does not split into separate per-agent distributions. Capturing the agent to agent and the agent to map relations requires representations able to operate on a variable and unordered number of entities, which is why structured formulations such as graphs and attention mechanisms prevail. Where these relations are modelled poorly, forecasts that appear plausible for each agent in isolation become inconsistent at the level of the scene comprising of multiple entities [2].

1.2.4 Real-World Constraints

Prediction is one stage of a control loop closing at a fixed rate, which leaves it a budget of tens of milliseconds on embedded hardware limited in memory and power. Its cost

grows with the number of participants and becomes quadratic once their interactions are modelled, so the least favourable case is dense traffic, precisely where prediction matters most. Model capacity therefore has to be weighed against computational efficiency, as accuracy alone does not make a model deployable [4, 2].

1.3 Thesis Outline

Chapter 2

Trajectory Prediction: Background and State of the Art

TODO: concrete header for the given thesis's theme!

2.1 Taxonomy of Trajectory Prediction Methods

In the literature, there are numerous different prediction methods, and several criteria enable their systematic classification, such as the modality and the representation of the input, the form of the produced output, or the paradigm by which prediction is integrated with perception. Here the modeling methods are classified according to the categorization presented in [6].

2.1.1 Physics-Based Methods

These rely on kinematic or dynamic models of motion, using physical equations to extrapolate future positions and not accounting for uncertainty. They are the oldest family in the field, resting on estimators formulated between the 1960s and the 2000s. Examples include constant-velocity and constant-acceleration models (Single-Trajectory simulation), applied to vehicle path prediction and to collision warning in the early 2000s [8, 9]. The Kalman filter, formulated in 1960 [10], addresses the uncertainty limitation of the previous models by modeling the vehicle's state as a Gaussian distribution. It predicts trajectories as a two step process of filtering (denoising current state from noisy measurements) and forecasting. Monte Carlo methods, and in particular the particle filter introduced in 1993 [11], drop the linearity and Gaussianity assumptions of the Kalman filter by approximating the state distribution with a set of weighted

samples propagated through the motion model, which permits arbitrarily shaped and multimodal distributions at the cost of a higher computational load. Physics-based approaches are simple and fast, but they struggle with complex maneuvers or sudden behavior changes since they lack contextual understanding. Some variants combine multiple motion models to handle different maneuvers (Multiple-Model methods), a family surveyed comprehensively in 2005 [12].

2.1.2 Learning-based methods

Unlike physics-based, learning-based methods learn the motion dynamics from data. Here, three groups of approaches are mentioned.

2.1.2.1 Classical Machine Learning Methods

These methods learn motion patterns from data using statistical models, but without deep neural networks. They were the dominant data-driven approach before deep learning took over, and they continued to appear in hybrid form well after it. The earliest representatives used here are Gaussian processs (GPs) applied to human motion in 2008 [13] and Gaussian mixture models (GMMs) used for probabilistic vehicle trajectory prediction in 2012 [14], followed by hidden Markov models (HMMs) for maneuver classification in 2018 [15] and by dynamic Bayesian networks (DBNs) combined with sequence learning in 2020 [16]. They can capture uncertainty to a degree (e.g., an HMM can model discrete maneuver states, GPs provide a distribution over trajectories). However, their capacity to model very complex, high-dimensional scenarios is limited compared to modern deep nets.

2.1.2.2 Reinforcement Learning Methods

Reinforcement learning (RL) methods treat prediction as a sequential decision problem. Here, an agent learns a policy (via reward feedback) to mimic human driving behavior, effectively planning likely trajectories. Since the reward that drives human driving is not known, inverse reinforcement learning (IRL) recovers it from expert demonstrations and uses it to regularise the predictor [17], while imitation learning (IL) skips the reward altogether and extracts the policy directly from the observed trajectories [18], both lines reaching trajectory prediction around 2020. RL-based predictors can naturally handle multi-agent interactions by modeling how agents might react to each other, and can ensure physically feasible outputs through learned policy constraints. However, RL methods can be data-hungry, harder to train and stabilize, and historically have been more common in planning than in pure trajectory forecasting. They are an active research area for capturing game-theoretic interactions and long-horizon planning in prediction.

2.1.2.3 Deep Learning Methods

TODO: add equations and figures to important parts

Various Deep Learning methods perform successfully in learning motion dynamics and now dominate trajectory prediction. The field moved through recurrent architectures around 2018, convolutional and graph-based ones between 2019 and 2021, and attention-based ones from 2021 onwards, each family addressing a limitation of the previous one. Their strengths and limitations differ mainly according to their architectural design.

Recurrent Neural Networks

Recurrent neural networks (RNNs) and long short-term memory (LSTM) capture temporal history via hidden states updated across timesteps. They are able to effectively capture temporal patterns for trajectory prediction task. Historically, single recurrent models were used to map past trajectories to future paths [19], however recent works facilitate groups or stacks of multiple models [20] or employ them for one of the subtasks like detection [21]. They converge reliably and can be trained autoregressively, in the same mode in which they are later run at inference, but they lack spatial reasoning and struggle with long-range temporal dependencies [22].

Convolutional Neural Networks

Convolutional neural networks (CNNs) have already succeeded in many tasks, most notably in computer vision [23]. They suit trajectory prediction because trajectories possess a strong spatio-temporal continuity, and because they operate on any input that can be arranged as a grid, which makes them applicable to the camera, the light detection and ranging (LiDAR) and the high-definition map (HD map) modalities alike. In every case they are responsible for feature extraction, capturing the positions of the objects and the environmental context around them. On camera images a convolutional backbone produces per-view features that are lifted into the bird's-eye view (BEV) plane through a predicted depth distribution [24]. On LiDAR point clouds the points are grouped into vertical pillars and flattened into a pseudo-image [25], from which a sparse convolutional backbone forms the BEV feature map [26]. HD maps are rasterised into additional channels of that same grid, so that lane geometry and traffic rules reach the network through the same convolutions as the sensor data [27].

Temporal convolutional networks (TCNs) are able to model temporal dependencies through the causal convolution layers that preserve the temporal order [28]. They extract spatial features from image-like inputs such as BEV or rasterised maps efficiently and enable for fast inference, although the fixed grid on which they operate represents the individual agents only implicitly and is more expensive than a vectorised encoding of

the same scene [22].

Graph Neural Networks

TODO: add graphnn from vip3d

Graph neural networks (GNNs) represent the scene as a graph whose nodes are the agents and map elements and whose edges encode the relations between them, which makes them a natural fit for modelling spatial and temporal interactions [29]. They excel in dense interaction scenarios by modelling agent-to-agent relations explicitly, but they add unnecessary complexity in sparse environments such as highway driving, and their dynamic-graph variants rely on consistent object identities and therefore inherit the errors of the tracker [6].

Transformers

Transformers replace recurrence with self-attention, which relates every agent and every map element to all the others in a single step and thus captures long-range dependencies without having to propagate information through a hidden state [30]. Since attending over all agents, time steps and map elements at once is expensive, practical models factorise the attention along these axes and decode the future with a set of learned queries that produce several trajectories in parallel [4]. Their capacity for long-term dependencies makes them particularly promising for long-horizon prediction, but they are highly sensitive to training stability and demand extensive tuning and warm-up [6]. Transformer systems pair a strong scene encoder with multi-modal decoders and explicit interaction and map context [30, 31].

2.2 Prediction Paradigms

In this section, the prediction paradigms are discussed. The prediction paradigm is an important characteristic of a model, as it determines how the driving stack is decomposed into modules, what is optimised during training, and what information is passed from perception to prediction. The advantages and the limitations of a given model largely follow from this choice.

2.2.1 Modular (Detect-Track-Predict)

This approach has been of the most common so far implemented in the literature. The paradigm consists of sequential steps of detection, tracking and prediction, which were independently under an extensive research. First, the sensor data is captured and detector localizes the objects present in the scene, usually as a set of bounding boxes with the corresponding class labels, either from LiDAR point clouds [32] or from camera images

[33]. The tracking module then associates these detections across consecutive frames [10], opening a tracklet for every newly observed object and preserving its identity as the scene evolves. Finally, the prediction module consumes the active tracklets and forecasts the future positions of the corresponding agents [6], commonly by encoding them together with the map in a vectorised form [34, 35]. Even though this sequential decomposition is intuitive and each module can be developed and debugged separately, the stages are optimised in isolation which causes errors to propagate across modules and results in suboptimal performance due to the lack of end-to-end optimization [7].

TODO: add picture

2.2.2 Joint Learning (End-to-End Perception & Prediction)

Joint learning methods treat detection and motion forecasting as a single optimization problem instead of two independent stages. These unified optimization goals help to mitigate the error accumulation in the intermediate steps. Interest in the paradigm has grown steadily since the first such models appeared in 2018 [36], to the point that a recent survey dedicated to end-to-end perception and prediction already reviews more than a hundred works [7]. Such models are trained with a compound loss over both tasks. They differ in how tightly the stages are coupled, ranging from a shared LiDAR backbone co-optimized through a combined loss [36], through BEV features reused by both tasks [37], to full gradient flow from the prediction loss back through the detector [38]. Query-based models take this integration further by representing every agent as a learnable query that is iteratively refined while attending to scene and map features, so that detection, tracking and forecasting are resolved together [39, 40]. The price of this coupling is that task boundaries blur, which makes poor performance hard to attribute to a specific component, and the iterative refinement of queries adds computational overhead that complicates real-time deployment in dense scenes [6].

TODO: add picture

TODO: add tree of the methods

2.3 Datasets

Research in trajectory prediction has been accelerated by large-scale autonomous driving datasets. They differ in the sensors they record, in the length and the annotation rate of their sequences, and in whether their annotations support perception, forecasting, or both.

- **KITTI (2012)** [41]: the early autonomous vehicle (AV) benchmark with stereo cameras, LiDAR and global positioning system (GPS)/inertial measurement unit

(IMU), widely used for detection, tracking and simultaneous localization and mapping (SLAM). With roughly 150 short sequences it supports only short-horizon forecasting and contains little social interaction.

- **ApolloScape (2018)** [42]: Chinese urban driving data covering detection, segmentation, tracking and mapping. It is useful for perception and localization, but it is less standardized for forecasting than nuScenes or Argoverse.
- **Lyft Level 5 (2019)** [43]: a large urban dataset with synchronized LiDAR and cameras and with HD maps, similar in format to nuScenes and used mostly for 3D detection and tracking.
- **nuScenes (2020)** [44]: 1,000 twenty-second scenes recorded with 6 cameras, 5 radars and one LiDAR covering 360°, annotated at 2 Hz with 3D boxes, HD maps and object identities that persist across frames. The official split is 700 training and 150 validation scenes, and the prediction challenge adds future trajectories over a 6 s horizon on exactly these sequences.
- **Argoverse 2 (2021)** [45]: the successor of the benchmark that popularized map-based prediction and established ADE, FDE and MR as the reported metrics [46]. It is published as several distinct collections, among them a motion forecasting set of 250k scenarios with per-scenario HD maps and a separate sensor set of 1,000 annotated sequences with seven ring cameras, two stereo cameras and LiDAR. The two collections contain different recordings.
- **Waymo Open Motion (2021)** [3]: more than 100k twenty-second scenarios sampled at 10 Hz, with HD maps and a large share of interactive situations. It introduced mAP for multi-modal predictions. Later releases of the dataset extend these scenarios with LiDAR point clouds [47] and with camera information, but both cover only the first second of each nine-second window, and the camera data is distributed as learned image tokens rather than as raw images.

NuScenes is used for every experiment reported in this thesis. A jointly trained perception and prediction model needs synchronized raw LiDAR and camera data, 3D boxes, object identities that persist across frames so that tracking can be supervised, and future trajectories, all annotated on the same sequences. Several of the datasets above hold these annotations apart. NuScenes defines an official prediction benchmark on exactly the sequences it annotates, and it is the dataset on which the end-to-end models discussed in Section 2.2 report their results, which makes the numbers obtained here directly comparable with them.

TODO: add table with datasets if training on multiple datasets, and also edit metrics if needed

2.4 Evaluation Metrics

A jointly trained model has to be judged on every task it solves, so three families of metrics are reported. The detection metrics measure how well the objects present in the scene are identified, the tracking metrics measure how reliably their identities are maintained over time, and the trajectory prediction metrics measure how well their future motion is anticipated. All three families are built on a notion of a correct match between a prediction and a ground-truth annotation, but they establish that match in different ways.

2.4.1 Detection Metrics

The protocol used throughout this thesis is the one defined with nuScenes [44] and summarized in [48]. A predicted box is matched to a ground-truth box of the same class when the distance between their centres on the ground plane is below a threshold d, which replaces the intersection over union criterion common in image-based detection and avoids penalising small objects disproportionately. Matching is *greedy* over predictions sorted by decreasing confidence, and every ground-truth box can be matched at most once. The outcome of this assignment defines the counts:

- a *true positive* is a prediction matched to a ground-truth box within threshold d,
- a *false positive* is a prediction that finds no such match, either because nothing is there or because the object was already claimed by a higher scoring prediction,
- a *true negative* is a correctly reported negative, absence of any detection.
- a *false negative* is a ground-truth box left without any matching prediction.

Based on these, the following performance metrics are defined:

1. **Precision and recall** express how many of the predictions are correct and how many of the annotated objects are found,

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}. \tag{2.1}$$

   Sweeping the *confidence score threshold* traces a precision-recall curve $p(r)$.

2. **Average Precision (AP)** summarises that curve for a given class as the area under its interpolated form, so that fluctuations of $p(r)$ at individual operating points do not influence the result [48],

$$AP = \int_0^1 \max\left\{ p(r) : r \geqslant r_0 \right\} dr_0, \tag{2.2}$$

   where $p(r)$ is the precision at recall r. The nuScenes protocol evaluates this over the operating points that remain once those with a recall or a precision below 0.1

have been discarded as dominated by noise, and normalises the result to the unit interval [44].

3. **Mean Average Precision (mAP)** averages AP over the set of classes C and over a set of matching thresholds $\mathcal{T}$, so that a method is not rewarded for being accurate only at one tolerance,

$$\mathrm{mAP} = \frac{1}{|C|\,|\mathcal{T}|} \sum_{c \in C} \sum_{t \in \mathcal{T}} \mathrm{AP}_{c,t}, \qquad \mathcal{T} = \{0.5,\, 1,\, 2,\, 4\}\,\mathrm{m}. \tag{2.3}$$

4. **True positive error metrics** describe how good the matched boxes actually are, which mAP alone does not capture because a match within d counts fully regardless of its quality. They are computed over the matches obtained at $\mathrm{d} = 2\,\mathrm{m}$ and averaged over classes. They comprise the **translation error ATE** as the centre distance in metres, the **scale error ASE** as $1 - \mathrm{IoU}$ after aligning translation and orientation, the **orientation error AOE** as the yaw difference in radians, the **velocity error AVE** in metres per second, and the **attribute error AAE** as one minus the attribute classification accuracy.
5. **NuScenes Detection Score (NDS)** combines the two into a single number, weighting the detection quality against the five error terms,

$$\mathrm{NDS} = \frac{1}{10}\left[\, 5\,\mathrm{mAP} + \sum_{\mathrm{mTP} \in \mathbb{TP}} \big(1 - \min(1,\, \mathrm{mTP})\big) \right], \tag{2.4}$$

where $\mathbb{TP}$ is the set of the five averaged error metrics. Each error is **clipped at one**, so an arbitrarily bad error cannot drive the score below the contribution of mAP alone.

2.4.2 Tracking Metrics

Tracking is as well evaluated with the protocol of the nuScenes tracking benchmark [44]. Predictions are matched to ground-truth objects frame by frame with the same centre distance criterion as in detection, but the assignment now also has to be consistent in time, because a tracker is required to keep the same identity attached to the same object. The matching therefore proceeds in two stages: a pairing established in an earlier frame is carried forward whenever both partners are still present and within the allowed distance, and only the objects and tracks left over are assigned by Hungarian matching on the centre distance. What is understood as a *true positive* therefore changes:

- a *true positive* is a ground-truth object matched within the allowed distance that either carried no track before or is matched to the track it already carried,
- a *false positive* is a track that is matched to no ground-truth object, so the tracker is

following something that does not exist,
- a *false negative* is a ground-truth object that no track reached in this frame,
- a *true negative* is a correctly reported negative, absence of any detection.
- an *identity switch (IDS)* is a ground-truth object matched to a track whose identity differs from the one it carried before. A track that is lost and later recovered under a new identity therefore produces an IDS even though every individual box may be accurate.

It is also important to state that the lifetime of a trajectory is not simply the set of frames in which it was annotated. The benchmark first discards annotations that fall outside the per-class evaluation range or that contain no LiDAR returns at all, and then linearly interpolates each trajectory across any remaining interior gap, so that a fully occluded object is restored. A trajectory's lifetime is therefore the uninterrupted span between its first and last surviving observation, and a tracker is expected to carry an identity through occlusions that the sensor itself did not observe. The tracking metrics are as follows:

1. **Multi-Object Tracking Accuracy (MOTA)** aggregates all the ways a tracker can fail into a single number, counting every false positive, every missed object and every identity switch against the number of annotated object instances GT summed over all frames,
$$\mathrm{MOTA} = 1 - \frac{\mathrm{FP} + \mathrm{FN} + \mathrm{IDS}}{\mathrm{GT}}. \tag{2.5}$$
The measure is not bounded from below, as a tracker that reports far more objects than exist can drive it to an arbitrarily negative value.
2. **Multi-Object Tracking Precision (MOTP)** describes how precisely the matched objects are localised, as the mean centre distance d_i over the set of true positives,
$$\mathrm{MOTP} = \frac{1}{|\mathrm{TP}|} \sum_{i \in \mathrm{TP}} d_i. \tag{2.6}$$
3. **Average Multi-Object Tracking Accuracy (AMOTA) and Average Multi-Object Tracking Precision (AMOTP)** remove the dependence on the confidence threshold in the same way AP does for detection. The tracking score is swept, and the two measures above are averaged over a set $\mathcal{R}$ of forty recall levels, from which those below 0.1 are discarded as dominated by noise,
$$\mathrm{AMOTA} = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} \mathrm{MOTAR}_r, \tag{2.7}$$
where MOTAR_r is the value of MOTA at recall r, normalised so that it occupies the unit interval at every recall level and is therefore comparable across them.

AMOTA is the primary measure of the benchmark.

4. **Supporting counts** describe the failure modes that the aggregate scores hide. Unlike MOTA, whose unit is a single object instance in a single frame, these are evaluated along the whole lifetime of one annotated object and each trajectory is counted once. A trajectory is reported as **mostly tracked (MT)** when it is followed for at least 80% of its lifetime and as **mostly lost (ML)** when it is followed for at most 20%. The remaining trajectories are **partially tracked (PT)**, so that the three categories partition the annotated trajectories,

$$N_{traj} = MT + PT + ML, \tag{2.8}$$

where each count refers to whole trajectories and N_{traj} are all annotated trajectories that survive the evaluation filters.
A **fragmentation** is counted whenever a correctly followed trajectory is interrupted and later resumed. The **track initialisation duration** expresses how long it takes before an object is picked up for the first time, and the **longest gap duration** how long it remains lost in the worst interruption.

These supporting counts matter for a model whose tracks are gated by a confidence threshold. A detector whose individual boxes are accurate but whose scores fluctuate around that threshold loses and reacquires its objects, which leaves AP almost unaffected while raising the number of fragmentations and identity switches and therefore lowering AMOTA.

2.4.3 Trajectory Prediction Metrics

The prediction metrics are computed over agents. An agent is characterised only by its position in the ground plane and its forecast trajectory. The predicted agents for a frame are assigned to the ground truth (GT) agents of that frame by Hungarian matching on the centre distance, with the same allowed distance of 2 m as in Section 2.4.2. The track identity is never taken into account, so the assignment is recomputed from scratch in every frame rather than carrying established pairings as in tracking. The confidence of a prediction is likewise not read, so the counts are evaluated at the single operating point at which the tracker emitted its tracks.

A matched agent is additionally only useful if its forecast is close to what actually happened, which is what the following metrics quantify. For an agent with K predicted future trajectories $\hat{y}^{(k)}$, a ground-truth future y and the set $\mathcal{T}$ of future time steps with a valid annotation:

1. **minADE** is the average displacement of the best of the K modes over the whole horizon, which rewards a model for containing one good hypothesis without

punishing it for the diversity of the others,

$$\text{minADE}_K = \min_{k\in\{1,\dots,K\}} \frac{1}{|\mathcal{T}|} \sum_{t\in\mathcal{T}} \left\| \hat{y}_t^{(k)} - y_t \right\|_2. \tag{2.9}$$

A single annotated future step is sufficient for an agent to contribute, and the average is taken over $|\mathcal{T}|$ rather than over the full horizon, so agents that leave the scene early are scored on a correspondingly shorter forecast.

2. **minFDE** evaluates only the endpoint of the horizon, where the uncertainty of the forecast is largest,

$$\text{minFDE}_K = \min_{k\in\{1,\dots,K\}} \left\| \hat{y}_T^{(k)} - y_T \right\|_2. \tag{2.10}$$

It is defined only for agents that are still annotated at the end of the horizon, that is $T \in \mathcal{T}$. An agent that disappears earlier is excluded entirely

3. **Miss Rate (MR)** turns the endpoint error into a binary outcome and reports how often none of the K modes lands near the true endpoint,

$$\text{MR} = \frac{\left|\{\, i : \text{minFDE}_K^{(i)} > \tau \,\}\right|}{N_{\text{surv}}}, \qquad \tau = 2\,\text{m}, \tag{2.11}$$

where the index i runs over the N_{surv} matched agents that survive the whole horizon, since the others have no minFDE to threshold.

4. **End-to-end Prediction Accuracy (EPA)** [39] is the metric that judges the two tasks jointly. An agent contributes to the numerator only if it was both matched and forecast accurately, and every false positive is subtracted with a penalty α,

$$\text{EPA} = \max\left(0,\ \frac{N_{\text{hit}} - \alpha\, N_{FP}}{N_{GT}}\right), \qquad \alpha = 0.5, \tag{2.12}$$

where N_{hit} counts the matched agents whose minFDE_K does not exceed τ, and N_{GT} the annotated agents that survive the whole horizon. Unlike the three metrics above, which are averaged over the agents that were found, EPA is normalised by the annotated agents rather than the matched ones, so *a model cannot improve it by detecting fewer objects and forecasting only the easy ones*.

The metrics are complementary and have to be read together. A model may reach a low minADE while missing many agents entirely, because the error is averaged only over the matched ones, and MR exposes the cases where no mode comes close at all. EPA penalises both failure modes at once and is therefore used as the primary criterion in this work.

Chapter 3

Related Work

Chapter 2 described the field as a whole. This chapter is narrower and covers only the works from which the model studied in this thesis is assembled. It begins with the query-based formulation that the model inherits, continues with the components introduced by the LiDAR branch, and closes with the two works that preceded this thesis in taking ViP3D as the base model and extending it with the LiDAR modality.

3.1 Query-Based Detection and Tracking

The architecture studied here descends from a line of work that removed many hand-designed components like non-maximum suppression (NMS) procedure or anchor generation. DETR [49] formulates detection as the direct prediction of a fixed-size set of object, in which a small number of learned queries attend to the image features and a Hungarian matching assigns every query to at most one annotated object.

The slow convergence caused by every query attending to every pixel was addressed by Deformable DETR [50], which lets a query sample only a few learned offsets around a reference point and thereby reduces both the cost and the number of epochs required for training. The reference point of a query is obtained by a linear projection of the positional part of its embedding, followed by a sigmoid that maps it into normalised coordinates. In the same work the authors introduce a two-stage variant that initialises the reference points from the centres of the highest scoring region proposals generated on top of the encoder output, so that the decoder starts from positions determined by the input rather than by a learned constant, and this variant outperforms the single-stage one [50]. DETR3D carried the formulation into three dimensions by placing the reference point of each query in 3D space and projecting it into the available camera views, so that multi-view images are consumed without an intermediate dense depth

estimate [33].

The step from detection to tracking was taken by MOTR, which keeps a query alive across frames instead of detecting the same object independently in each one. The identity of an object is then carried by the query itself rather than recovered afterwards by an association heuristic [51]. How reliably that identity survives across frames is what the tracking metrics of Section 2.4.2 measure. ViP3D combines the two ideas and extends them to forecasting. Agent queries are initialised in the DETR3D manner, propagated through time in the MOTR manner by a memory bank and a query interaction module, and read directly by the trajectory decoder. The result is a pipeline that is differentiable from the camera images to the predicted trajectories and that contains no thresholded intermediate representation [39]. ViP3D is the model this thesis builds upon.

3.2 LiDAR Backbones and Heatmap Query Initialisation

ViP3D perceives the scene through cameras alone, so its queries are initialised from a fixed learned embedding that is identical in every frame regardless of what the scene contains. Replacing that embedding requires a representation of the point cloud and a way of reading object hypotheses out of it.

The representation follows the established encoders described in Section 2.1.2.3. PointPillars groups the points into vertical columns and flattens them into a pseudo-image, which allows a conventional two-dimensional backbone to process the point cloud at a cost that does not depend on its density [25], and SECOND forms the BEV feature map with sparse convolutions that skip the empty regions of the grid [26].

The hypotheses come from CenterPoint, which abandons anchors and instead predicts a class-wise heatmap over the BEV grid, where every peak marks the centre of an object [32]. TransFusion [52] turned this into the initialisation of a transformer decoder. *The highest scoring heatmap peaks become object queries whose content is sampled from the BEV feature* at the peak and whose reference point is the position of that cell, so that the decoder begins from a hypothesis that depends on the input rather than from a constant embedding. *This is the mechanism that replaces the learned query embedding of ViP3D in the present work.*

3.3 Attention-Based Camera-LiDAR Fusion

After the queries originate from the point cloud 3.2, the camera modality may be introduced to supply the colour and texture that the point cloud does not contain. TransFusion [52] performs this both in the initialization procedure and by adding a

second decoder layer that cross-attends to the image features by projecting the queries' location into image plane like [50]. It trains that layer on top of a frozen LiDAR model, which prevents the fusion from destabilising a branch that was already pretrained beforehand. The attention in that layer is spatially modulated following SMCA [53], which adds the logarithm of a Gaussian centred on the projection of the reference point of the query to the attention logits before the softmax, so that a query attends only where it can geometrically be and converges considerably faster than unconstrained cross-attention.

The initialisation itself operates in the BEV plane rather than in the image. TransFusion collapses the features of each image along the vertical axis, so that one image column becomes a single token, on the observation that a BEV location maps to a column by camera geometry and that a column rarely holds more than one object. The BEV cells then attend to those columns, the result is averaged over the cameras, and a second heatmap predicted from it is combined with the LiDAR one, so that the camera takes part in deciding which queries are created at all.

3.4 Trajectory Decoding from Agent Features

The last stage turns the state of an agent into a set of possible futures. VectorNet replaced the rasterised scene encodings common at the time with a vectorised one, in which agent histories and map elements are represented as polylines and a hierarchical graph aggregates first within and then across them [34]. LaneGCN refined the treatment of the map by building an explicit lane graph and exchanging information between the agents and the lanes in both directions [35]. The decoder of ViP3D follows this line, with one substantial difference. It does not receive an observed trajectory assembled by a tracker, but the feature vector of the agent query itself, which still carries the visual evidence that a bounding box would have discarded [39].

3.5 Positioning of This Work

The idea of giving ViP3D a LiDAR branch has been pursued before in the group in which this thesis was written. Li-ViP3D [54] addresses limited capability of exploitation of the complementarity between camera and LiDAR data in the current perception and prediction (PnP) solutions by extending ViP3D query-based camera-only pipeline with camera-LiDAR fusion and reports improved end-to-end perception and prediction on nuScenes. Li-ViP3D++ [55] develops the fusion further by gating the contribution of the camera on the query and by sampling the image features in deformable manner. Both of these works treat the addition of LiDAR as a fusion problem, in which the two

modalities are combined for queries that are still generated the way ViP3D generates them from a learned embedding that never depends on any particular scene.

The present work asks where the queries should come from and tackles the problem of their initialisation. As mentioned in 3.1, Deformable DETR [50] indroduced a two-stage variant initialises the reference points from the centres of the highest scoring region proposals generated on top of the encoder output, so that the decoder starts from positions determined by the input. This variant outperformed the single-stage one, which is evidence that reference points derived from the data are prefereable to reference points learned as constants. In this thesis, the same principle was applied in the BEV plane. The proposal generator is a class-wise heatmap predicted from the LiDAR feature map, and its highest scoring peaks supply the query with both the reference point from the position of the peak and the content from the feature sampled at that cell.

This work is therefore positioned differently from the two that precede it. Li-ViP3D and Li-ViP3D++ combine the two modalities for queries that are still created the way ViP3D creates them, so the LiDAR enters at the moment a query is refined. Here it enters earlier, at the moment a query is created, and the camera is reintroduced only afterwards in a second decoder layer trained on top of the frozen LiDAR model. Fusion is thus a refinement of a hypothesis that the point cloud has already formed.

3.6 Summary and Starting Points

Based on the study presented in Chapters 2 and 3, the following gaps are identified.

- **Gap 1:** query-based PnP models create their queries from a learned embedding that is identical in every frame, although the detection literature has shown that reference points derived from the input outperform reference points learned as constants. Input-dependent query initialisation has therefore not been applied in end-to-end perception and prediction, where a query must not only localise an object but also carry its identity across frames.
- **Gap 2:** a query that already begins at a position read from the point cloud has little left to correct, with consequences for decoder depth, for inference latency and for how much the camera still contributes. These effects have not been investigated in the tracking and prediction tasks.

The goal of this thesis is therefore to develop and evaluate a deep learning architecture for end-to-end perception and prediction in which the agent queries originate in the point cloud, extending ViP3D with a LiDAR branch in place of its learned query embedding.

To fill in the gaps, the following starting points are proposed.

- **Starting point 1:** replace the learned query embedding of ViP3D with features extracted from LiDAR BEV using a class-wise heatmap predicted from the BEV feature map, taking the position of the cell as an initial position of the query.
- **Starting point 2:** introduce a shallower decoder to exploit the fact that the queries already begin near their objects. Compare a decoder given only LiDAR with one that reintroduces the camera, and measure the performance differences.

Priebežná správa BP1/DP1 musí obsahovať:

1. Prehľad podobných riešení alebo produktov, s ktorými sa študent oboznámil pri štúdiu danej problematiky.
2. Špecifikáciu požiadaviek na vytvárané riešenie alebo produkt (Chapter 4).
3. Návrh
4. Vlastne zhodnotenie práce za príslušné obdobie (Chapter C - Section C.1).

Chapter 4

Objectives and Methodology

Chapters 2 and 3 identified two gaps, namely that query-based PnP models as ViP3D [39] create their agent queries from a learned embedding that does not depend on the scene, and that the consequences of changing this have not been measured in tracking and prediction. This chapter states the objectives that follow and the methodology by which they are pursued.

4.1 Objectives

The goal of this thesis is to develop and evaluate a deep learning architecture for end-to-end (E2E) perception and prediction in which the agent queries originate in the point cloud and not in a learned embedding. The goal is pursued through three objectives.

- **Objective 1.** Design and implement a query initialisation mechanism that derives the agent queries from the LiDAR BEV feature map, and integrate it with the tracker inside ViP3D [39] codebase so that queries carried over from the previous frame keep their slots and the initialisation concerns newly appearing objects alone. The resulting model needs to evaluated next to other representants of E2E trajectory prediction.
- **Objective 2.** Determine what the camera modality still contributes once the queries already carry geometric evidence by implementing multilayer architecture for modality fusion.
- **Objective 3.** Determine what a query that already begins near its object buys in decoder depth and in inference latency emphasizing the trade-off between prediction accuracy and computational cost.

Objective 1 addresses the first gap. Objectives 2 and 3 address the second gap mentioned

in 3.6.

4.2 Methodology

The work follows the research cycle of analysis, design, implementation, experimental evaluation and interpretation. Each step is carried by one chapter of this thesis. A brief description of the steps is provided in this section.

- **Step 1, analysis.** Survey trajectory prediction and 3D object detection literature, with attention to how query-based models initialize their agent queries. The outcome is the two gaps and the starting points stated in Section 3.6. Carried by Chapters 2 and 3.
- **Step 2, design.** Propose an architecture that closes those gaps, deciding how a query is created from the point cloud, at which point the camera modality re-enters and how the training is staged. Designing it mainly concerns designing the initialization mechanisms, design of the fusion decoder layers, design of integration with the ViP3D codebase and finally scheduling the training according to the identified literature. Carried by Chapter 5.
- **Step 3, implementation.** Realise the design by extending an existing open implementation of ViP3D [39]. Keeping the surrounding code fixed is a methodological choice, since it makes the proposed change the only difference between the baseline and the proposed model. This way direct comparison can be achieved. Carried by Chapter 6.
- **Step 4, experimental evaluation.** Measure the implemented model against selected E2E PnP models from the literature, and isolate the contribution of each added component by ablating it, one ablation per objective. Detection, tracking and prediction metrics are recorded for every configuration and reported per class, together with inference latency and parameter count. Chapter 7.
- **Step 5, interpretation.** Given large number of experiments needed to conduct, proper interpretation of the results is crucial. The results are analysed using metrics provided in Chapter 2.4 and interpreted in the context of the objectives and the research questions for clear conclusions and recommendation for future work. Carried by both Chapter 7 and Chapter 8.

This chapter derived three objectives from the gaps identified in Chapter 3 and set out the research cycle through which they are pursued, from the analysis that produced the gaps to the interpretation of the results. In the following chapter, the design of the proposed model is described in detail.

Chapter 5

Design of the Proposed Model

This chapter describes the design of the proposed model, referred to as InitViP3D in what follows, at the level of its components, the data that passes between them and the algorithms they execute. Its realisation in a particular library is left to Chapter 6. InitViP3D extends ViP3D [39] and keeps most of its pipeline, so the chapter first describes ViP3D in full and then the parts that this thesis replaces or adds, which follow the starting points of Section 3.6. It closes with the training schedule and an estimate of the time and memory complexity of every stage.

5.1 Requirements

The requirements follow from the gaps and the starting points of Section 3.6. These functional requirements state what the model has to do.

- **FR1.** Every query slot that *does not follow an object* is initialised from the current LiDAR observation, with its reference point placed where an object is likely and its content read from the BEV features at that position.
- **FR2.** A query that already follows an object *keeps its state from the previous frame*, and no new query is placed where such an object lies, so that the initialisation concerns newly appearing objects alone.
- **FR3.** The *camera contributes through a fusion layer* that refines queries already formed from the point cloud, and the model remains functional without that layer.
- **FR4.** Detection, tracking and trajectory prediction are *produced by one network* from one set of queries in end-to-end manner, as in ViP3D.

The following non-functional requirements constrain how it has to be implemented.

- **NFR1 (Experimental control).** Every part of ViP3D *outside the query path is kept unchanged*, so that a difference in the results can be attributed to the proposed

change.

- **NFR2 (Architectural Efficiency).** The decoder structure shall *support reduced layer depths* (fewer than the baseline six layers) to evaluate inference speed and parameter efficiency when queries are initialized near target locations.
- **NFR3 (Modularity & Trainability).** The camera fusion layer shall be *trainable independently* while freezing the pre-trained LiDAR feature extractor weights, ensuring the camera stream can be added on top of a fixed geometric representation.

5.2 Overview

InitViP3D processes a scene frame by frame and carries a set of agent queries from one frame to the next. One frame passes through five stages.

1. **Feature extraction.** The point cloud is encoded into a BEV feature map and, when the camera is used, the six images into multi-level image features.
2. **Query initialisation.** A class-wise heatmap is predicted over the BEV map, and its strongest peaks initialise the query slots that do not follow an object.
3. **Decoding.** A two-layer decoder refines all queries, the first layer by reading the BEV map and the second by attending to the camera features, and after each layer predicts a box, a class and a velocity for every query.
4. **Tracking update.** Queries are assigned identities, their embeddings are refined by a memory of their own past and by interaction with the other tracks, and the reference points of the surviving tracks are moved into the coordinate frame of the next frame.
5. **Trajectory prediction.** The embeddings of the tracked agents, together with the lanes of an HD map, are decoded into several future trajectories per agent.

Stages 1, 4 and 5 are those of ViP3D. Stage 2 replaces the learned query embedding of ViP3D, and stage 3 replaces its six-layer camera decoder.

The symbols used throughout the chapter are listed in Table 5.1, together with the values they take in the configuration used for the experiments.

5.3 The ViP3D Baseline

This section describes ViP3D as it is used as the baseline and as the frame into which the proposed components are inserted. Its general idea was introduced in Section 3.4, so the focus here is on how its parts operate and exchange data.

Table 5.1: Symbols used in this chapter and their values in the configuration used.

Symbol	Meaning	Value
N_q	empty query slots appended in every frame	300
N_a	track queries carried over from the previous frame	varies
$Q = N_q + N_a$	queries processed by the decoder in one frame	varies
D	embedding width	256
N_{cls}	object classes	7
$H \times W$	BEV grid	256×256
s	BEV cell size	0.4 m
C	channels of the BEV feature map	384
N_{cam}	cameras	6
$h \times w$	camera feature map read by the fusion layer	68×120
T	frames in a training clip	3
L	length of the memory bank	4
M	predicted trajectories per agent	6
T_f	predicted future positions, 6 s at 2 Hz	12

5.3.1 Agent Queries and Track State

The central object of ViP3D is the agent query. Following MOTR [51], a frame is processed with two kinds of queries. Track queries are carried over from the previous frame and follow objects that have already been found, while a fixed number N_q of empty slots is appended in every frame to find objects that are new. The decoder therefore processes $Q = N_q + N_a$ queries in a frame, where N_a is the number of tracks alive at that moment. Each slot i holds a state

$$\mathcal{S}_i = (\mathbf{p}_i,\, \mathbf{c}_i,\, \mathbf{r}_i,\, \mathbf{z}_i,\, \mathbf{h}_i,\, \mathrm{id}_i,\, s_i,\, \mathbf{B}_i)\,, \tag{5.1}$$

where $\mathbf{p}_i \in \mathbb{R}^D$ and $\mathbf{c}_i \in \mathbb{R}^D$ are the positional and the content half of the query, $\mathbf{r}_i \in \mathbb{R}^3$ is its reference point, $\mathbf{z}_i \in \mathbb{R}^3$ is the logarithm of the width, length and height of its box, $\mathbf{h}_i \in \mathbb{R}^D$ is the output embedding of the last decoder layer, id_i is the identity of the object the slot follows or -1 for an empty slot, s_i is its score, and $\mathbf{B}_i \in \mathbb{R}^{L \times D}$ is a memory bank of its past embeddings.

The reference point is not stored in metres. Its coordinates are first normalised to the point cloud range $[x_{min}, x_{max}] \times [y_{min}, y_{max}] \times [z_{min}, z_{max}]$ and then passed through the inverse of the sigmoid function σ,

$$\mathbf{r}_i = \sigma^{-1}(\tilde{\mathbf{r}}_i)\,, \qquad \tilde{r}_{i,x} = \frac{x_i - x_{min}}{x_{max} - x_{min}}, \tag{5.2}$$

and likewise for y and z. Every refinement of the reference point is then an unbounded offset added to $\mathbf{r}_i$, and applying σ afterwards maps the result back into the range, so that a refined point can never leave it.

In ViP3D an empty slot always starts from the same values. Both halves of its query are taken from a learned embedding $\mathbf{E} \in \mathbb{R}^{N_q \times 2D}$, its reference point is predicted from the positional half by a linear layer, $\mathbf{r}_i = \mathbf{W}_r \mathbf{p}_i$, and its initial box size by another linear layer from the same half. None of these depends on the scene, which is the property this thesis changes.

5.3.2 Camera Feature Extraction

Each of the N_{cam} images is encoded by a ResNet-50 [56], whose last two stages use deformable convolutions [57], followed by an feature pyramid network (FPN) [58] that produces four feature maps of D channels at strides 8, 16, 32 and 64 of the input image. Before the decoder reads them, each map receives a sine positional encoding of its pixel coordinates, a learned embedding of its pyramid level and a learned embedding of its camera, so that every feature carries where in which image it was computed.

5.3.3 Six-Layer Decoder with Projective Cross-Attention

The decoder of ViP3D follows DETR3D [33] and consists of six identical layers. Each layer applies self-attention [59] among all Q queries, then cross-attention to the image features, then an feed-forward network (FFN), each followed by a residual connection and layer normalisation.

The cross-attention does not attend densely. The reference point of a query is projected into every camera with the projection matrix of that camera, every pyramid level is sampled bilinearly at the projected location, and the samples are combined with weights predicted from the query, while samples from projections that fall behind the camera or outside the image are masked out. A query therefore reads one point per camera and pyramid level, at most $4N_{cam}$ feature vectors per layer, and relies on six layers of refinement to move its reference point onto the object. After every layer the reference point and the box size are refined as described in Section 5.6.3.

5.3.4 Prediction Branches

Every decoder layer has its own set of four prediction branches applied to its output embedding. The classification branch is an multilayer perceptron (MLP) of two hidden layers that outputs N_{cls} logits, and the three regression branches are MLPs of the same depth that output six geometric offsets, the sine and cosine of the heading, and the velocity (v_x, v_y). Together they give the box code

$$\mathbf{b} = \left(c_x,\, c_y,\, \log w,\, \log l,\, c_z,\, \log h,\, \sin\theta,\, \cos\theta,\, v_x,\, v_y\right). \tag{5.3}$$

The centre is predicted as an offset from the reference point in inverse-sigmoid space, $\hat{c}_x = \sigma(\Delta c_x + r_x)$ and likewise for y and z, and is then scaled back to metres. The size is predicted as an offset in logarithmic space and added to the size carried by the slot, $\hat{\mathbf{z}} = \mathbf{z} + \Delta\mathbf{z}$. The class score of a slot is the maximum sigmoid probability over the classes, $s_i = \max_k \sigma(\hat{y}_{i,k})$, taken from the last decoder layer. Every layer is supervised separately, so that each layer learns to produce a usable prediction rather than only the last one.

5.3.5 Tracking

Tracking decides which slot follows which object, keeps the embedding of a followed object informed about its past, and moves its query into the next frame. It is realised differently in training and at inference.

Identity assignment in training. During training the identity of a slot is taken from the GT, as in MOTR. A slot that follows an object keeps it as long as the object is annotated, and only the empty slots compete for objects that no slot follows yet. The competition is resolved by the Hungarian method [60] with the cost

$$C(i,n) = \lambda_{\text{cls}}\, C_{\text{focal}}(\hat{\mathbf{y}}_i, k_n) + \lambda_{\text{L1}} \left\| \hat{\mathbf{b}}_i^{(1:8)} - \mathbf{b}_n^{(1:8)} \right\|_1, \tag{5.4}$$

where C_{focal} is the focal classification cost of predicting class k_n of object n, the box term compares the first eight components of the box code, velocity excluded, and $\lambda_{\text{cls}} = 2$ and $\lambda_{\text{L1}} = 0.25$. The procedure is given in Algorithm 1. It is run for each decoder layer separately to supervise that layer, but only the assignment made on the last layer is kept as the identity of the slot.

Track birth and death at inference. At inference no annotation is available, and identities are governed by the score of each slot, as given in Algorithm 2. An empty slot whose score reaches $\tau_{\text{new}} = 0.4$ becomes a new track. A track whose score falls below $\tau_{\text{keep}} = 0.35$ accumulates a miss, and after $m_{\max} = 5$ consecutive misses it is ended. Between the two thresholds a track neither resets nor accumulates misses, so the gate has a hysteresis. Only tracks with a score of at least τ_{keep} are reported in the output. These thresholds sit behind hard comparisons, so no gradient reaches them, and they are inherited from ViP3D unchanged.

Algorithm 1 Identity assignment and supervision targets for one frame during training

Require: slots with identities id_i and predictions $(\hat{\mathbf{y}}_i, \hat{\mathbf{b}}_i)$, annotated objects $\mathcal{G}_t$ of frame t with persistent identities
Ensure: target g_i of every slot, an object of $\mathcal{G}_t$ or none

for every slot i with $\mathrm{id}_i \geqslant 0$ **do**
  if an object of $\mathcal{G}_t$ has identity id_i **then**
    $g_i \leftarrow$ that object ▷ inherited, no matching needed
  else
    $g_i \leftarrow$ none ▷ object left the scene, trained as background
  end if
end for
$\mathcal{E} \leftarrow \{i : \mathrm{id}_i = -1\}$ ▷ empty slots
$\mathcal{U} \leftarrow$ objects of $\mathcal{G}_t$ that no slot follows ▷ new or reappearing
$\pi \leftarrow \textsc{Hungarian}(C(\mathcal{E}, \mathcal{U}))$ ▷ cost of Eq. (5.4)
for every pair $(i, n) \in \pi$ **do**
  $\mathrm{id}_i \leftarrow$ identity of n, $\quad g_i \leftarrow n$ ▷ track birth
end for
classification loss on all slots, positive where g_i is an object
box loss on the slots whose g_i is an object

Algorithm 2 Track birth and death at inference

Require: slots with scores s_i, identities id_i and miss counters m_i

for every slot i **do**
  if $s_i \geqslant \tau_{\mathrm{new}}$ **then**
    $m_i \leftarrow 0$
  end if
  if $\mathrm{id}_i = -1$ **and** $s_i \geqslant \tau_{\mathrm{new}}$ **then**
    $\mathrm{id}_i \leftarrow$ next unused identity ▷ track birth
  else if $\mathrm{id}_i \geqslant 0$ **and** $s_i < \tau_{\mathrm{keep}}$ **then**
    $m_i \leftarrow m_i + 1$
    if $m_i \geqslant m_{\max}$ **then**
      $\mathrm{id}_i \leftarrow -1$ ▷ track death
    end if
  end if
end for

Memory bank. Every track keeps its last L stored embeddings in $\mathbf{B}_i$. Once a track has at least one stored entry, its current output embedding $\mathbf{h}_i$ attends to them with multi-head attention, followed by a residual connection, layer normalisation and an FFN, so that the embedding of a track is refined by its own history. A projection of the refined embedding is then pushed into the bank and the oldest entry dropped. During training every frame is stored, while at inference a track stores an entry only every fourth frame, so at test time the bank spans a longer stretch of the past than it does in training.

Query interaction module. The query interaction module (QIM) updates the queries of the tracks that survive into the next frame. The active tracks attend to each other through self-attention, with the sum of their positional half and output embedding as queries and keys and their output embedding as values, followed by an FFN. Two further FFNs map the result to updates of the positional half and of the content half of the query, each added with a residual connection and normalised. The query of a track thus carries what the decoder saw of it in this frame into the next one. During training the module also drops each active track with probability 0.1 and adds, for each active track with probability 0.3, one of the empty slots with the highest score, so that the model learns to recover a lost track and to reject a false one.

Propagation into the next frame. Before a track is decoded in the next frame, its reference point is moved by its own predicted velocity and by the motion of the ego vehicle. With $(\mathbf{R}_t, \mathbf{t}_t)$ the pose of the LiDAR frame at time t in global coordinates and Δt the time between the frames, the metric reference point $\mathbf{x}_t$ becomes

$$\mathbf{x}_{t+1} = \mathbf{R}_{t+1}^{\top} \left(\mathbf{R}_t \left(\mathbf{x}_t + \mathbf{v}_t \, \Delta t \right) + \mathbf{t}_t - \mathbf{t}_{t+1} \right), \tag{5.5}$$

where $\mathbf{v}_t = (v_x, v_y, 0)$ is the velocity predicted by the last decoder layer. The result is normalised and passed through σ^{-1} as in Eq. (5.2). When Δt lies outside $[0, 1]$ s, as happens at the start of a new scene, only the ego transformation is applied.

5.3.6 Trajectory Prediction

Trajectory prediction follows VectorNet [34] with the agent and lane interaction of LaneGCN [35]. The lanes and lane connectors of the HD map within 70 m of the ego vehicle are discretised at 1 m, each becomes a polyline of vectors, and a polyline subgraph of self-attention layers and max-pooling encodes each polyline into one vector.

The agents are not described by their past trajectories. Their representation is the output embedding of their track query, which already holds what the decoder and the memory bank gathered about the agent, so that nothing is lost at the interface between tracking

and prediction. When camera features are available, this embedding is first refined by one more projective camera cross-attention at the last box centre of the agent. It then passes two MLP layers, and the predictor reads its first 128 components, which is the working width of the predictor. The lanes attend to the lanes and the agents, a global self-attention graph relates all agents and lanes to each other, and a decoder outputs for every agent M trajectories of T_f positions in the coordinate frame of the agent, together with a probability π_m for each of them.

The predictor is trained with a variety loss. For an agent a whose last valid future position is at step τ_a, only the mode closest to the GT at that step,

$$m^* = \arg\min_m \left\| \hat{\mathbf{y}}^{a,m}_{\tau_a} - \mathbf{y}^a_{\tau_a} \right\|_2 , \tag{5.6}$$

is regressed, and the probability of that mode is pushed up,

$$\mathcal{L}_{\text{pred}} = \frac{1}{A} \sum_{a=1}^{A} \left(\frac{1}{|\mathcal{V}_a|} \sum_{t \in \mathcal{V}_a} \text{SmoothL1}\left(\hat{\mathbf{y}}^{a,m^*}_t - \mathbf{y}^a_t \right) - \log \pi^a_{m^*} \right), \tag{5.7}$$

where $\mathcal{V}_a$ is the set of valid future steps of agent a and A the number of supervised agents. Regressing only the closest mode lets the modes specialise to different futures instead of collapsing onto their mean. Only tracks matched to an annotated agent with a valid future are supervised.

5.4 LiDAR Branch

The point cloud of the current frame is accumulated with the nine preceding sweeps, so that ten sweeps enter the branch, and every point carries five features, its three coordinates, its intensity and its time offset from the current sweep. The branch follows PointPillars [25] and the convolutional backbone of SECOND [26], as introduced in Section 3.2.

Space within ±51.2 m around the vehicle is divided into pillars of 0.2×0.2 m that span the whole vertical range, and a pillar holds at most 20 points. A PointNet-style encoder [61] turns the points of each non-empty pillar into a vector of 64 features, and the vectors are scattered back to the positions of their pillars, which gives a pseudo-image of 512×512 cells. A two-dimensional CNN with two strided stages reduces it to 256×256 cells with 128 channels and to 128×128 cells with 256 channels, and a neck upsamples both to 256×256 and concatenates them. The result is the BEV feature map

$$\mathbf{F} \in \mathbb{R}^{C \times H \times W}, \qquad C = 384, \quad H = W = 256, \quad s = 0.4\,\mathrm{m}. \tag{5.8}$$

The map serves twice. The heatmap head predicts object centres from it, and the first decoder layer reads features from it at the reference points of the queries.

5.5 Query Initialisation from the LiDAR Heatmap

This section describes the first contribution of the thesis, which replaces the learned embedding of ViP3D by queries derived from the point cloud, as proposed in Starting point 1. The principle was introduced in Section 3.2, and what follows is its design inside a tracking model.

5.5.1 Class-Wise Heatmap Head

The heatmap head predicts, for every class and every BEV cell, the probability that the centre of an object of that class lies in the cell. A shared 3×3 convolution reduces the BEV map to 64 channels, followed by batch normalisation and a rectified linear unit. Several small heads then each predict a group of classes, with the classes grouped as in CenterPoint [32] so that similar shapes share a head. Each head consists of a 3×3 convolution with normalisation and activation and a final 3×3 convolution with one output channel per class. The outputs are reordered into the class order of the model, which gives the logits $\hat{\mathbf{Y}} \in \mathbb{R}^{N_{\mathrm{cls}} \times H \times W}$. The bias of the final convolutions is initialised so that every cell starts with a probability of about 0.1, which keeps the loss stable in the first iterations when almost every cell is background.

The target heatmap is built from the annotated boxes. An object n of class k_n with centre (x_n, y_n) and footprint $w_n \times l_n$ falls into the cell (i_n, j_n) with $j_n = \lfloor (x_n - x_{\min})/s \rfloor$ and $i_n = \lfloor (y_n - y_{\min})/s \rfloor$, and is drawn as a Gaussian whose radius follows its size,

$$\begin{aligned} Y_{k,i,j} &= \max_{n:\, k_n = k} \exp\!\left(-\frac{(i - i_n)^2 + (j - j_n)^2}{2\sigma_n^2} \right), \\ \sigma_n &= \frac{2\rho_n + 1}{6}, \qquad \rho_n = \max\!\left(1, \left\lceil \frac{\sqrt{w_n^2 + l_n^2}}{2s} \right\rceil \right), \end{aligned} \tag{5.9}$$

evaluated within a window of $2\rho_n + 1$ cells around the centre. The radius ρ_n is half the diagonal of the footprint in cells, so a bus spreads over more cells than a pedestrian, and the maximum keeps the peak of every object at exactly 1 where objects of the same class overlap. The head is trained with the Gaussian focal loss of CornerNet [62],

$$\mathcal{L}_{\text{hm}} = -\frac{1}{N_{\text{pos}}} \sum_{k,i,j} \begin{cases} \left(1 - \hat{P}_{k,i,j}\right)^{\alpha} \log \hat{P}_{k,i,j}, & Y_{k,i,j} = 1, \\ \left(1 - Y_{k,i,j}\right)^{\beta} \hat{P}_{k,i,j}^{\alpha} \log\left(1 - \hat{P}_{k,i,j}\right), & \text{otherwise}, \end{cases} \tag{5.10}$$

where $\hat{\mathbf{P}} = \sigma(\hat{\mathbf{Y}})$, N_{pos} is the number of cells with $Y_{k,i,j} = 1$, and $\alpha = 2$ and $\beta = 4$. The factor $(1 - Y)^{\beta}$ lowers the penalty on cells close to a centre, so the head is not punished for a peak that lands one cell off. Normalising by N_{pos} makes the loss an average over objects instead of a sum over the whole grid.

5.5.2 Selecting Peaks and Building Queries

The empty slots of a frame are filled by Algorithm 3. The heatmap probabilities are first detached from the computation graph, since the selection of a cell is a discrete choice through which no gradient can pass. Cells around every active track are then set to zero within a window of 5×5 cells, which is $2 \times 2\,\text{m}$, so that a new query cannot be born on an object that a track already follows. A 3×3 max-pooling keeps only the local maxima of every class, which acts as a cheap non-maximum suppression. It is skipped for bicycles and pedestrians, whose footprints are smaller than the pooling window and which often stand closer to each other than one window, so that suppression would remove real objects. The strongest cells are finally selected jointly over all classes and all positions, so that the classes compete for the slots according to their scores.

A selected cell (i, j) of class k becomes a query in two steps. Its reference point is the metric centre of the cell,

$$x = x_{\min} + \left(j + \tfrac{1}{2}\right) s, \qquad y = y_{\min} + \left(i + \tfrac{1}{2}\right) s, \qquad z = \tfrac{1}{2}\left(z_{\min} + z_{\max}\right), \tag{5.11}$$

normalised and passed through σ^{-1} as in Eq. (5.2). The height is set to the middle of the range, since the BEV map carries no vertical position, and is left for the decoder to refine. Its content is the BEV feature of the cell, projected to the embedding width, plus a learned embedding of the class,

$$\mathbf{c} = \mathbf{W}_{\text{bev}}\, \mathbf{F}_{:,i,j} + \mathbf{e}_k, \qquad \mathbf{p} = \mathbf{c}, \tag{5.12}$$

where $\mathbf{W}_{\text{bev}} \in \mathbb{R}^{D \times C}$ and $\mathbf{e}_k \in \mathbb{R}^{D}$. The positional half is set equal to the content half at birth, since the content already encodes where the cell is, and the two halves diverge as soon as the query interaction module updates them.

Algorithm 3 Initialisation of the empty slots from the LiDAR heatmap

Require: heatmap logits $\hat{\mathbf{Y}} \in \mathbb{R}^{N_{cls} \times H \times W}$, BEV map $\mathbf{F}$, slots $\mathcal{S}$
Ensure: the empty slots of $\mathcal{S}$ receive a reference point and a query

function InitQueries($\hat{\mathbf{Y}}, \mathbf{F}, \mathcal{S}$)
  $\mathbf{P} \leftarrow \text{stopgrad}\big(\sigma(\hat{\mathbf{Y}})\big)$
  for every slot $a \in \mathcal{S}$ with $\text{id}_a \geqslant 0$ **do**
    $(i_a, j_a) \leftarrow$ BEV cell of the reference point $\mathbf{r}_a$
    $\mathbf{P}_{:,\, i_a-2:i_a+2,\, j_a-2:j_a+2} \leftarrow 0$ ▷ no birth on a tracked object
  end for
  $\mathbf{P}^{\max} \leftarrow \text{maxpool}_{3\times3}(\mathbf{P})$
  for every class k except bicycle and pedestrian **do**
    $\mathbf{P}_k \leftarrow \mathbf{P}_k \odot \mathbf{1}\big[\mathbf{P}_k = \mathbf{P}_k^{\max}\big]$ ▷ keep local maxima only
  end for
  $\mathcal{E} \leftarrow \{i \in \mathcal{S} : \text{id}_i = -1\}$ ▷ empty slots
  $\{(k_n, i_n, j_n)\}_{n=1}^{|\mathcal{E}|} \leftarrow$ the $|\mathcal{E}|$ largest entries of $\mathbf{P}$ over all classes and cells
  for $n = 1, \ldots, |\mathcal{E}|$ **do**
    $(x, y, z) \leftarrow$ metric centre of cell (i_n, j_n) ▷ Eq. (5.11)
    $\mathbf{r}_{\mathcal{E}_n} \leftarrow \sigma^{-1}\big(\text{normalise}(x, y, z)\big)$
    $\mathbf{c} \leftarrow \mathbf{W}_{\text{bev}}\, \mathbf{F}_{:, i_n, j_n} + \mathbf{e}_{k_n}$
    $(\mathbf{p}_{\mathcal{E}_n}, \mathbf{c}_{\mathcal{E}_n}) \leftarrow (\mathbf{c}, \mathbf{c})$
  end for
end function

5.5.3 Integration with the Tracker

The initialisation is inserted into ViP3D at the single place where an empty slot used to receive its learned values, and it touches no other part of the tracking. Four properties of the integration follow from that placement.

- **Only empty slots are filled.** A slot with an identity keeps the query and the reference point it carried from the previous frame, so a followed object is never re-initialised and keeps its history. The suppression around active tracks complements this by keeping new queries off the cells those tracks occupy.
- **Birth is still decided by the tracker.** Placing a query on a peak does not create a track. In training the slot must still win the matching of Algorithm 1, and at inference its score must still reach τ_{new} in Algorithm 2. A weak peak therefore produces a query that the decoder can reject.
- **Gradients reach the initialisation only through the content.** The choice of cells is discrete, so the heatmap head learns only from its own loss of Eq. (5.10). The detection loss reaches the BEV features, the projection $\mathbf{W}_{\text{bev}}$ and the class embeddings through the content of Eq. (5.12), which teaches them to describe an object in a way the decoder can use.
- **The initial box size is a learned constant.** The size of a new slot is not derived from its class, so every new query starts from the same learned size, which the first decoder layer refines before the second layer needs it.

Slots that the query interaction module adds as false positives during training keep the identity -1, so in the next frame they are re-initialised from the heatmap like any other empty slot. The complete processing of one frame at inference is summarised in Algorithm 4.

Algorithm 4 Processing of one frame by InitViP3D at inference

Require: point cloud $\mathcal{P}_t$, images $\{I_t^m\}_{m=1}^{N_{\text{cam}}}$, tracks $\mathcal{T}_{t-1}$, ego poses of frames $t-1$ and t
Ensure: tracks $\mathcal{T}_t$ with boxes, classes and identities

move the reference points of $\mathcal{T}_{t-1}$ into frame t ▷ Eq. (5.5)
$\mathcal{S} \leftarrow \mathcal{T}_{t-1} \cup \{N_q \text{ empty slots}\}$
$\mathbf{F} \leftarrow \textsc{LidarBranch}(\mathcal{P}_t)$, $\quad \{\mathbf{F}^m\} \leftarrow \textsc{CameraBranch}(\{I_t^m\})$
$\hat{\mathbf{Y}} \leftarrow \textsc{HeatmapHead}(\mathbf{F})$
$\textsc{InitQueries}(\hat{\mathbf{Y}}, \mathbf{F}, \mathcal{S})$ ▷ Algorithm 3
decode all slots of $\mathcal{S}$ with layer 0 and layer 1 ▷ Section 5.6
update scores and identities ▷ Algorithm 2
refine the embeddings of the slots with the memory bank
$\mathcal{T}_t \leftarrow$ the slots with id $\geqslant 0$, their queries updated by the QIM
return $\mathcal{T}_t$

5.6 Two-Layer Fusion Decoder

This section describes the second contribution, the shallow decoder of Starting point 2. It replaces the six camera layers of ViP3D with two layers of different kinds. The first reads the BEV map and the second attends to the camera features, which follows the decoder of TransFusion [52] described in Section 3.3. Both layers share the structure of a transformer decoder layer, namely self-attention among all queries, cross-attention to a sensor, and an FFN with a hidden width of 512, each followed by a residual connection and layer normalisation, with eight attention heads throughout.

5.6.1 Layer 0: Reading the BEV Map

The cross-attention of the first layer has the form of deformable attention [50] with $P = 4$ sampling points per query and learned weights over them. In the configuration used, the learned sampling offsets are disabled, so all four points coincide with the reference point of the query and the layer reduces to a bilinear read of the BEV map at that point,

$$\mathbf{o}_i = \mathbf{W}_o \, \mathbf{W}_{in} \, \mathbf{F}\big(\tilde{r}_{i,x}, \tilde{r}_{i,y}\big) \,, \tag{5.13}$$

where $\mathbf{F}(\cdot)$ denotes bilinear interpolation of the map at the normalised position, $\mathbf{W}_{in} \in \mathbb{R}^{D \times C}$ projects the BEV channels to the embedding width and $\mathbf{W}_o$ is an output projection. The output is added to the query as a residual. The reasoning behind this simplification is that a new query is born on a heatmap peak and a track has just been moved onto its predicted position, so in both cases the reference point already lies on the object and the feature at that point is the evidence the query needs. Reading a fixed number of points also makes the cost of the layer independent of the size of the BEV grid.

5.6.2 Layer 1: Spatially Modulated Camera Attention

The second layer reintroduces the camera through spatially modulated co-attention (SMCA) [53], as proposed in Starting point 2. Unlike the projective cross-attention of ViP3D, which reads one point per camera, it attends to every position of a camera feature map, and a Gaussian centred on the projection of the reference point decides how much each position may contribute. It reads only the finest pyramid level, at stride 8, whose map has $h \times w$ positions per camera.

For each camera m, the reference point is projected into the image, which gives its position (u_m, v_m) on the feature map and a flag v_m that holds when the point lies in front of the camera and inside the image. The width of the Gaussian follows the size

of the box in the image. The centre of the box and the four points at half its width and half its length along the axes are projected, the extent they cover on the feature map is measured, and

$$\sigma_m = \mathrm{clip}\left(\beta \cdot \tfrac{1}{2}\sqrt{\Delta u_m^2 + \Delta v_m^2},\ 1,\ 50\right), \qquad G_m(u,v) = \exp\left(-\frac{(u-u_m)^2 + (v-v_m)^2}{\sigma_m^2}\right), \tag{5.14}$$

where Δu_m and Δv_m are the horizontal and vertical extent in feature-map positions, $\beta = 1$ is a scale, and only points in front of the camera are used to measure the extent. A nearby car covers many positions and receives a wide Gaussian, a distant one a narrow Gaussian. The attention of each head adds the logarithm of the Gaussian to the logits before the softmax,

$$\mathbf{A}_m = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}_m^\top}{\sqrt{d_h}} + \log \mathbf{G}_m\right), \qquad \mathbf{o}_m = \mathbf{A}_m \mathbf{V}_m, \tag{5.15}$$

where $\mathbf{Q}$ is projected from the queries, $\mathbf{K}_m$ and $\mathbf{V}_m$ from the features of camera m, and $d_h = D/8$. Since $\log G_m = 0$ at the projected centre and falls quadratically away from it, a position far from the object is excluded however similar its features are. The outputs are averaged over the cameras that see the query,

$$\mathbf{o} = \frac{\sum_m \nu_m \mathbf{o}_m}{\max(1, \sum_m \nu_m)}, \tag{5.16}$$

projected, and added to the query together with an encoding of the three-dimensional reference point produced by a small MLP. A query that no camera sees receives nothing from Eq. (5.16), and for such queries the whole output of the layer is replaced by the output of layer 0. Objects outside the field of view of every camera, or close to the vehicle below the lower image edges, are therefore described by the LiDAR alone rather than by an update computed from no evidence. The computation is summarised in Algorithm 5.

5.6.3 Reference Refinement

After each decoder layer l, the regression branch of that layer predicts an offset of the reference point and of the box size, and both are updated before the next layer,

Algorithm 5 Spatially modulated cross-attention of decoder layer 1

Require: queries $\mathbf{X}$, reference points $\mathbf{r}$, box sizes $\mathbf{z}$, camera features $\mathbf{F}^1, \dots, \mathbf{F}^{N_{cam}}$
Ensure: updated queries and the set of queries seen by no camera

function SMCA($\mathbf{X}, \mathbf{r}, \mathbf{z}, \{\mathbf{F}^m\}$)
  $\mathbf{o} \leftarrow \mathbf{0}, \quad \mathbf{n} \leftarrow \mathbf{0}$
  $\mathbf{Q} \leftarrow \mathbf{W}_Q \mathbf{X}$ split into 8 heads
  for $m = 1, \dots, N_{cam}$ **do**
    $(u_m, v_m, \nu_m) \leftarrow$ projection of the reference points into camera m
    $\sigma_m \leftarrow$ width from the projected box extent ▷ Eq. (5.14)
    $\mathbf{G}_m \leftarrow$ Gaussian over the $h \times w$ positions
    $\mathbf{K}_m, \mathbf{V}_m \leftarrow \mathbf{W}_K \mathbf{F}^m, \ \mathbf{W}_V \mathbf{F}^m$
    $\mathbf{A}_m \leftarrow \mathrm{softmax}\left(\mathbf{Q}\mathbf{K}_m^\top / \sqrt{d_h} + \log \mathbf{G}_m\right)$
    $\mathbf{o} \leftarrow \mathbf{o} + \nu_m \mathbf{A}_m \mathbf{V}_m, \quad \mathbf{n} \leftarrow \mathbf{n} + \nu_m$
  end for
  $\mathbf{o} \leftarrow \mathbf{W}_o \, \mathbf{o} / \max(1, \mathbf{n})$
  return $\mathbf{X} + \mathbf{o} + \mathrm{MLP}\left(\sigma^{-1}(\tilde{\mathbf{r}})\right), \quad \{i : n_i = 0\}$
end function

$$\tilde{\mathbf{r}}^{(l+1)} = \mathrm{stopgrad}\Big(\sigma\Big(\sigma^{-1}\big(\tilde{\mathbf{r}}^{(l)}\big) + \Delta\mathbf{r}^{(l)}\Big)\Big), \qquad \mathbf{z}^{(l+1)} = \mathrm{stopgrad}\Big(\mathbf{z}^{(l)} + \Delta\mathbf{z}^{(l)}\Big), \tag{5.17}$$

where $\tilde{\mathbf{r}}^{(l)} \in [0, 1]^3$ is the normalised reference point entering layer l, so the offset is added in inverse-sigmoid space and the result is mapped straight back into the range.

The stop-gradient keeps each layer responsible for its own prediction. A layer is supervised on the correction it makes, and its error is not propagated back into the layer that supplied its starting point, which stabilises training in the same way as in Deformable DETR. The refinement of layer 0 matters particularly here, since layer 1 uses the refined reference point to place its Gaussian and the refined size to set its width. The reference point produced by the last layer is the one that is propagated into the next frame.

5.6.4 Comparison with the Decoder of ViP3D

Table 5.2 sets the two decoders side by side. The reduction from six layers to two is possible because the queries no longer start from constants, and the work of moving a query onto its object is done before the decoder by the heatmap. The camera layer is, however, more expensive than a layer of ViP3D, since it attends densely to whole feature maps instead of sampling one point per camera. The saving in depth and the cost of the dense attention are therefore not independent, and Section 5.9 estimates both.

Table 5.2: The decoder of ViP3D compared with the decoder of InitViP3D.

	ViP3D	InitViP3D
Initialisation of empty slots	learned embedding, the same in every frame	strongest heatmap peaks of the current frame
Supervision before decoding	none	heatmap loss
Number of layers	6	2
LiDAR	not used	read at the reference point in layer 0
Camera	projective sampling, one point per camera and level	dense attention over one level, modulated by a Gaussian
Query seen by no camera	sampled features masked	output of layer 0 kept

5.7 Camera Visibility for Partial Camera Rigs

The six cameras of nuScenes cover the full circle around the vehicle. When the model is configured with fewer cameras, objects behind the vehicle remain annotated but leave no trace in any image, and a camera model would be trained to detect what it cannot see. Two masks prevent this, and both are active only when fewer than six cameras are used. In training, an annotated box whose centre projects into no camera keeps its place in the matching, so that the number of objects stays consistent, but its classification and box losses are set to zero. In the selection of Algorithm 3, every BEV cell is projected into the cameras, and cells that no camera sees are set to zero before the peaks are taken, so that queries are born only where a camera can confirm them. With six cameras both masks are left out, since they would otherwise also remove objects close to the vehicle that the LiDAR sees perfectly well.

5.8 Training

5.8.1 Loss

The model is trained on clips of T consecutive frames, so that identities can be carried from one frame to the next and the tracking modules receive a learning signal. The loss of a clip is

$$\mathcal{L} = \sum_{t=1}^{T} \left(\sum_{l=0}^{1} \left(\lambda_{\text{cls}} \mathcal{L}_{\text{cls}}^{t,l} + \lambda_{\text{box}} \mathcal{L}_{\text{box}}^{t,l} \right) + \lambda_{\text{hm}} \mathcal{L}_{\text{hm}}^{t} \right) + \mathcal{L}_{\text{pred}}, \tag{5.18}$$

where $\mathcal{L}_{\text{cls}}^{t,l}$ is the sigmoid focal loss [63] with $\gamma = 2$ and $\alpha = 0.25$ on the classification of all slots of frame t after layer l, positive for slots assigned to an annotated object, and $\mathcal{L}_{\text{box}}^{t,l}$ is the L1 loss between the predicted and the annotated box codes of Eq. (5.3) over the assigned slots, with the velocity components weighted by 0.2. The heatmap term is that of Eq. (5.10) and the prediction term that of Eq. (5.7). The weights are $\lambda_{\text{cls}} = 2$, $\lambda_{\text{box}} = 0.25$ and $\lambda_{\text{hm}} = 1$. Each detection and heatmap term is normalised within its own frame, the classification and box terms by the number of assigned slots and the heatmap term by the number of peaks, so that a crowded frame does not outweigh a quiet one.

5.8.2 Two-Stage Schedule

Training follows the two-stage recipe of TransFusion [52], which satisfies NFR3.

- **Stage 1, LiDAR only.** The camera input is absent. Layer 1 is bypassed and its outputs are copies of those of layer 0, so the second set of prediction branches is already trained on a meaningful representation when the camera layer is introduced. Everything else is trained, namely the LiDAR branch, the heatmap head, layer 0, both sets of prediction branches, the tracking modules and the predictor.
- **Stage 2, camera fusion.** Starting from the end of stage 1, the LiDAR feature extractor is frozen and the camera layer is trained from its initialisation. The image backbone, pretrained for detection, is frozen as well, and the image neck is fine-tuned at a tenth of the learning rate. The heatmap head is fine-tuned at a tenth of the learning rate, while layer 0, the prediction branches, the tracking modules and the predictor continue to train at the full rate.

Freezing the LiDAR feature extractor means that the camera cannot degrade the geometric representation that the queries are built from. Fusion becomes a refinement of hypotheses that the point cloud has already formed, as intended in Section 3.5. Algorithm 6 summarises the processing of one training clip.

5.9 Complexity Analysis

Table 5.3 estimates the time and memory complexity of each stage for one frame. Time is given as the number of multiply and accumulate operations, and the orders of magnitude are computed from the layer shapes of the configuration of Table 5.1, with $Q \approx 300$. They estimate the arithmetic of each stage and are not measured run times, which Chapter 7 reports.

Three observations follow from the table.

Algorithm 6 Training on one clip

Require: clip of T frames with sensor data, annotated boxes with identities and future trajectories, stage $\in \{1, 2\}$
Ensure: loss $\mathcal{L}$ of the clip

$\mathcal{S} \leftarrow N_q$ empty slots
for $t = 1, \ldots, T$ **do**
    extract $\mathbf{F}$, and the camera features in stage 2
    $\hat{\mathbf{Y}} \leftarrow \textsc{HeatmapHead}(\mathbf{F})$, add $\lambda_{hm}\mathcal{L}_{hm}^{t}$
    $\textsc{InitQueries}(\hat{\mathbf{Y}}, \mathbf{F}, \mathcal{S})$
    decode with layer 0, and with layer 1 in stage 2 or a copy of layer 0 in stage 1
    for $l = 0, 1$ **do**
        assign targets to the predictions of layer l ▷ Algorithm 1
        add $\lambda_{cls}\mathcal{L}_{cls}^{t,l} + \lambda_{box}\mathcal{L}_{box}^{t,l}$
    end for
    keep the identities assigned on layer 1
    memory bank, then QIM with random drop and false-positive insertion
    if $t < T$ **then**
        move the reference points into frame $t + 1$ ▷ Eq. (5.5)
    end if
    $\mathcal{S} \leftarrow$ surviving tracks $\cup \{N_q$ empty slots$\}$
end for
predict trajectories of the tracks matched to annotated agents, add $\mathcal{L}_{pred}$
return $\mathcal{L}$

Table 5.3: Estimated time and memory complexity of one frame. N_{pts} is the number of points, $H_p W_p$ the pillar grid, k the convolution kernel, hw the positions of one camera feature map, N_a the active tracks, n the size of the assignment problem, and A and M_l the agents and lanes of the predictor. Orders of magnitude refer to the configuration used.

Stage	Time	Order of time	Memory
Pillar encoding, scatter	$O(N_{pts})$	10^7	$O(H_p W_p)$
LiDAR CNN and neck	$O(HWC^2k^2)$	10^{11}	$O(HWC)$
Camera backbone and FPN	$O(N_{cam} H_{img} W_{img})$	10^{11} to 10^{12}	$O(N_{cam} hwD)$
Heatmap head	$O(HWCk^2)$	10^9	$O(N_{cls} HW)$
Query selection	$O(N_{cls} HW + N_a)$	10^6	$O(N_{cls} HW)$
Self-attention, per layer	$O(Q^2 D)$	10^7	$O(Q^2)$
Layer 0, BEV read	$O(QPCD)$	10^8	$O(QPC)$
Layer 1, SMCA	$O(N_{cam}(Q\,hwD + hwD^2))$	10^{10}	$O(Q\,N_{cam}\,hw)$
Prediction branches	$O(QD^2)$	10^8	$O(QD)$
Memory bank	$O(N_a LD)$	10^6	$O(N_a LD)$
Query interaction	$O(N_a^2 D)$	10^6	$O(N_a^2)$
Hungarian matching, training only	$O(n^3)$	10^7	$O(n^2)$
Trajectory prediction	$O((A + M_l)^2 D)$	10^7	$O((A + M_l)^2)$

The query path is cheap. The selection of queries from the heatmap is linear in the size of the grid and costs no more than a pooling and a partial sort, and the read of layer 0 is linear in the number of queries and independent of the grid. The contribution of this thesis to the query path therefore adds almost nothing to the cost of a frame. The heatmap head costs more, but it is a few convolutions over a map that the backbone has already produced.

The feature extractors dominate the arithmetic. The two convolutional backbones account for the largest share of the operations. The camera backbone is the larger of the two, since it processes six images, and it is absent in stage 1, which makes the LiDAR-only model considerably cheaper than the fused one independently of the decoder.

The camera layer dominates the decoder and its memory. Removing four layers saves little, since a layer of ViP3D samples only a few points per query. The SMCA layer instead forms, for each camera, an attention matrix of $8 \times Q \times hw$ entries, about 2×10^7 values, and a Gaussian mask of $Q \times N_{cam} \times hw$ entries, about 1.5×10^7 values. These are the largest intermediate tensors of the decoder, and since the layer moves much data for comparatively little arithmetic, its run time is limited by memory traffic rather than by the operation count of the table. The shallower decoder is therefore not cheaper

by construction, and whether it is faster than the six layers of ViP3D is a question for measurement.

Communication. At inference the model runs on a single device and exchanges no data. Training is data-parallel, so each step exchanges the gradients of the trainable parameters among the devices, which is $O(|\theta|)$ per step and fewer parameters in stage 2, where the feature extractors are frozen.

5.10 Summary

InitViP3D keeps the agent queries, the tracking modules and the trajectory predictor of ViP3D, and changes how a query begins and how it is refined. Empty slots are no longer filled from a learned constant but from the strongest peaks of a class-wise heatmap predicted from the LiDAR BEV map, with suppression around active tracks so that initialisation concerns new objects only. The six camera layers of ViP3D are replaced by two layers, one reading the BEV map at the reference point and one attending to the cameras through spatially modulated attention, with the LiDAR output kept for queries that no camera sees. The model is trained in two stages, the LiDAR model first and the camera layer on top of the frozen LiDAR feature extractor second. The realisation of this design is described in the following chapter.

Chapter 6

Implementation

Concrete header for the implementation of the proposed design

The concrete realization of our proposed design

Concrete tools, software, solutions based on concrete dataset

Pseudocode of Algorithms, or Code snippets

$$\text{Bounce Rate} = \frac{\text{Single page sessions}}{\text{Total sessions}} \tag{6.1}$$

Chapter 7

Experiments and Evaluation

7.1 Setup

hardware,
software, libraries, versioning,
virtual environment description and setting,
datasets, NDA,
etc.

Machine	Remote	Google Colab	Kaggle	FIIT Cloud	Local
Python	3.9.19	3.10.12	-	-	3.10.12
PyTorch	-	-	-	-	-
TensorFlow	2.10	2.15	-	-	2.14
GPU (NVIDIA)	RTX xx	Tesla T4 15 GB	Tesla P100 16GB	-	-
CUDA	12.2	12.2	-	-	-
cuDNN	8.1.0	N/A	-	-	-
CUDA Toolkit	11.2.2	N/A	-	-	-
RAM	32/64 GB	51 GB	30 GB	32 GB	16 GB
CPU	8x cores Intel i7	-	Xeon 4x cores	4x cores	-
Storage	SSD	Google Driver	73 GB	128 GB	HDD

Table 7.1: Hardware, software and versioning used for realization

7.2 Case Study - Experiment 01

7.3 Case Study - Experiment 02

7.4 Case Study - Experiment 03

7.5 Summary

Main *key findings* of the work based on the Experiment results

- add ...
- ...

Discussion about various considerations based on the experiment results (if needed).

Regarding *limitations* of the work ...

- add ...
- ...

Chapter 8

Conclusion

8.1 Summary

Summary of the whole work including SOTA, design, implementations, experiments, evaluations, key fingdings ... 1 page

8.2 Future Work

1/2 page

Resumé

Chapter je bez cislovania, pouziva sa len **boldovanie textu** na zyraznenie useku textu. Bez Figure, Table, atd. ak treba tak odvolava sa na originalne Figure a Table.

10% v slovenčine pre tých, ktorí majú prácu v EN: preklad

EN	SK
Introduction	primerane strucne
SOTA	primerane strucne
SOTA - cast Summary and Starting Points	cele
Objective and Methodology	cele
Design, Implementation	primerane strucne
Experiments and Evaluation	primerane strucne
Experiments and Evaluation - cast Summary	cele
Conclusion and Future Work	cele

Table 8.1: Resumé

List of Abbreviations

ADE Average Displacement Error

AMOTA Average Multi-Object Tracking Accuracy

AMOTP Average Multi-Object Tracking Precision

AP Average Precision

AV autonomous vehicle

AVs autonomous vehicles

BEV bird's-eye view

CNN convolutional neural network

DBN dynamic Bayesian network

E2E end-to-end

EPA End-to-end Prediction Accuracy

FDE Final Displacement Error

FFN feed-forward network

FPN feature pyramid network

GMM Gaussian mixture model

GNN graph neural network

GP Gaussian process

GPS global positioning system

GT ground truth

HD map high-definition map

HMM hidden Markov model

IDS identity switch

IL imitation learning

IMU inertial measurement unit

IoU intersection over union

IRL inverse reinforcement learning

LiDAR light detection and ranging

LSTM long short-term memory

mAP mean Average Precision

MLP multilayer perceptron

MOTA Multi-Object Tracking Accuracy

MOTP Multi-Object Tracking Precision

MR Miss Rate

NDS nuScenes Detection Score

NMS non-maximum suppression

PnP perception and prediction

QIM query interaction module

RL reinforcement learning

RNN recurrent neural network

SLAM simultaneous localization and mapping

SMCA spatially modulated co-attention

TCN temporal convolutional network

List of Figures

A.1 The structure of the ZIP file B-2

List of Tables

5.1 Symbols used in this chapter and their values in the configuration used. 27
5.2 The decoder of ViP3D compared with the decoder of InitViP3D. 40
5.3 Estimated time and memory complexity of one frame. N_{pts} is the number of points, $H_p W_p$ the pillar grid, k the convolution kernel, hw the positions of one camera feature map, N_a the active tracks, n the size of the assignment problem, and A and M_l the agents and lanes of the predictor. Orders of magnitude refer to the configuration used. 43

7.1 Hardware, software and versioning used for realization 47

8.1 Resumé 51

References

[1] Li Chen et al. "End-to-end autonomous driving: Challenges and frontiers". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.12 (2024), pp. 10164–10183. DOI: 10.1109/TPAMI.2024.3435937.

[2] Yanjun Huang et al. "A Survey on Trajectory-Prediction Methods for Autonomous Driving". In: *IEEE Transactions on Intelligent Vehicles* 7.3 (2022), pp. 652–674. DOI: 10.1109/TIV.2022.3167103.

[3] Scott Ettinger et al. "The Waymo Open Motion Dataset: A Large-Scale Benchmark for Motion Forecasting". In: *Proceedings of the Conference on Robot Learning (CoRL)*. 2021.

[4] Nigamaa Nayakanti et al. *Wayformer: Motion Forecasting via Simple & Efficient Attention Networks*. 2022. arXiv: 2207.05844 [cs.CV]. URL: https://arxiv.org/abs/2207.05844.

[5] *Road vehicles — Safety of the intended functionality (SOTIF)*. International Organization for Standardization, 2022.

[6] Nadya Abdel Madjid et al. "Trajectory Prediction for Autonomous Driving: Progress, Limitations, and Future Directions". In: *arXiv preprint arXiv:2503.03262* (2025). URL: https://arxiv.org/abs/2503.03262.

[7] Yufan Hu et al. "A Survey on End-to-End Perception and Prediction for Autonomous Driving". In: *Machine Intelligence Research* 22.6 (2025), pp. 999–1030. DOI: 10.1007/s11633-025-1558-0.

[8] Chiu-Feng Lin, A. Galip Ulsoy, and David J. LeBlanc. "Vehicle Dynamics and External Disturbance Estimation for Vehicle Path Prediction". In: *IEEE Transactions on Control Systems Technology* 8 (2000), pp. 508–518. URL: https://api.semanticscholar.org/CorpusID:33770615.

[9] Rick Miller and Qingfeng Huang. "An Adaptive Peer-to-Peer Collision Warning System". In: *IEEE 55th Vehicular Technology Conference (VTC Spring)*. Vol. 1. 2002, pp. 317–321. URL: https://api.semanticscholar.org/CorpusID:18526419.

[10] Rudolf E. Kalman. "A New Approach to Linear Filtering and Prediction Problems". In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45. DOI: 10.1115/1.3662552.

[11] Neil J. Gordon, David J. Salmond, and Adrian F. M. Smith. "Novel approach to nonlinear/non-Gaussian Bayesian state estimation". In: *IEE Proceedings F (Radar and Signal Processing)* 140.2 (1993), pp. 107–113. DOI: 10.1049/ip-f-2.1993.0015.

[12] X. Rong Li and Vesselin P. Jilkov. "Survey of Maneuvering Target Tracking. Part V: Multiple-Model Methods". In: *IEEE Transactions on Aerospace and Electronic Systems* 41 (2005), pp. 1255–1321. DOI: 10.1109/TAES.2005.1561886.

[13] Jack M. Wang, David J. Fleet, and Aaron Hertzmann. "Gaussian Process Dynamical Models for Human Motion". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 30 (2008), pp. 283–298. DOI: 10.1109/TPAMI.2007.1167.

[14] Jürgen Wiest et al. "Probabilistic Trajectory Prediction with Gaussian Mixture Models". In: *2012 IEEE Intelligent Vehicles Symposium*. 2012, pp. 141–146. DOI: 10.1109/IVS.2012.6232277.

[15] Nachiket Deo, Akshay Rangesh, and Mohan M. Trivedi. "How Would Surround Vehicles Move? A Unified Framework for Maneuver Classification and Motion Prediction". In: *IEEE Transactions on Intelligent Vehicles* 3 (2018), pp. 129–140. URL: https://api.semanticscholar.org/CorpusID:9228629.

[16] Yuxiao Li et al. "Pedestrian Trajectory Prediction Combining Probabilistic Reasoning and Sequence Learning". In: *IEEE Transactions on Intelligent Vehicles* 5 (2020), pp. 461–474. DOI: 10.1109/TIV.2020.2966117.

[17] Dooseop Choi, Kyoungwook Min, and Jeongdan Choi. "Regularising Neural Networks for Future Trajectory Prediction via Inverse Reinforcement Learning Framework". In: *IET Computer Vision* 14 (2020), pp. 192–200. DOI: 10.1049/iet-cvi.2019.0546.

[18] Mengshi Qi et al. "Imitative Non-Autoregressive Modeling for Trajectory Forecasting and Imputation". In: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 12733–12742. DOI: 10.1109/CVPR42600.2020.01275.

[19] M. Hoy et al. "Learning to Predict Pedestrian Intention via Variational Tracking Networks". In: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. 2018, pp. 3132–3137. DOI: 10.1109/ITSC.2018.8569641.

[20] L. Xin et al. "Intention-Aware Long Horizon Trajectory Prediction of Surrounding Vehicles Using Dual LSTM Networks". In: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE Press, 2018, pp. 1441–1446.

[21] Hao Shao et al. "ReasonNet: End-to-End Driving with Temporal and Global Reasoning". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023.

[22] Jianbang Liu et al. "A Survey on Deep-Learning Approaches for Vehicle Trajectory Prediction in Autonomous Driving". In: *2021 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. 2021. DOI: 10.1109/ROBIO54168.2021.9739407.

[23] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25. 2012, pp. 1097–1105.

[24] Jonah Philion and Sanja Fidler. "Lift, Splat, Shoot: Encoding Images From Arbitrary Camera Rigs by Implicitly Unprojecting to Bird's-Eye View". In: *European Conference on Computer Vision (ECCV)*. 2020, pp. 194–210.

[25] Alex H. Lang et al. "PointPillars: Fast Encoders for Object Detection from Point Clouds". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 12689–12697. DOI: 10.1109/CVPR.2019.01298.

[26] Yan Yan, Yuxing Mao, and Bin Li. "SECOND: Sparsely Embedded Convolutional Detection". In: *Sensors* 18.10 (2018), p. 3337. DOI: 10.3390/s18103337.

[27] Tung Phan-Minh et al. "CoverNet: Multimodal Behavior Prediction Using Trajectory Sets". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 14062–14071. DOI: 10.1109/CVPR42600.2020.01408.

[28] N. Nikhil and B. T. Morris. "Convolutional Neural Network for Trajectory Prediction". In: *Computer Vision – ECCV 2018 Workshops*. Ed. by L. Leal-Taixé and S. Roth. Cham: Springer International Publishing, 2019, pp. 186–196.

[29] C. Wang, S. Cai, and G. S. H. Tan. "GraphTCN: Spatio-Temporal Interaction Modeling for Human Trajectory Prediction". In: *2021 IEEE Winter Conference on Applications of Computer Vision (WACV)*. 2021, pp. 3449–3458. URL: https://api.semanticscholar.org/CorpusID:212725547.

[30] Yicheng Liu et al. "Multimodal Motion Prediction with Stacked Transformers". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, pp. 7573–7582. DOI: 10.1109/CVPR46437.2021.00749.

[31] Jiquan Ngiam et al. "Scene Transformer: A Unified Architecture for Predicting Multiple Agent Trajectories". In: *International Conference on Learning Representations (ICLR)*. 2022. URL: https://openreview.net/forum?id=Kjk%5C_H1To6A.

[32] Tianwei Yin, Xingyi Zhou, and Philipp Krähenbühl. "Center-Based 3D Object Detection and Tracking". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.

[33] Yue Wang et al. "DETR3D: 3D Object Detection from Multi-View Images via 3D-to-2D Queries". In: *Proceedings of the 5th Conference on Robot Learning (CoRL)*. 2021.

[34] Jiyang Gao et al. "VectorNet: Encoding HD Maps and Agent Dynamics from Vectorized Representation". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 11525–11533.

[35] Ming Liang et al. "Learning Lane Graph Representations for Motion Forecasting". In: *European Conference on Computer Vision (ECCV)*. 2020.

[36] Wenjie Luo, Bin Yang, and Raquel Urtasun. "Fast and Furious: Real Time End-to-End 3D Detection, Tracking and Motion Forecasting with a Single Convolutional Net". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 3569–3577. DOI: 10.1109/CVPR.2018.00376.

[37] Anthony Hu et al. "FIERY: Future Instance Prediction in Bird's-Eye View from Surround Monocular Cameras". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 15273–15282.

[38] Ming Liang et al. "PnPNet: End-to-End Perception and Prediction with Tracking in the Loop". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 11553–11562. DOI: 10.1109/CVPR42600.2020.01157.

[39] Junru Gu et al. "ViP3D: End-to-End Visual Trajectory Prediction via 3D Agent Queries". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023, pp. 5496–5506.

[40] Sergio Casas et al. "DeTra: A Unified Model for Object Detection and Trajectory Forecasting". In: *European Conference on Computer Vision (ECCV)*. 2024. URL: https://api.semanticscholar.org/CorpusID:270357489.

[41] Andreas Geiger, Philip Lenz, and Raquel Urtasun. "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2012, pp. 3354–3361.

[42] Xiaoyong Huang et al. "ApolloScape: 3D Car Understanding from Monocular Images". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2018.

[43] R. Kesten et al. "Level 5 AV Dataset 2019". In: *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2019.

[44] Holger Caesar et al. "nuScenes: A Multimodal Dataset for Autonomous Driving". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020.

[45] Benjamin Wilson, William Qi, et al. "Argoverse 2: Next Generation Datasets for Self-Driving Perception and Forecasting". In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. 2021.

[46] Ming-Fang Chang et al. "Argoverse: 3D Tracking and Forecasting With Rich Maps". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.

[47] Kan Chen, Runzhou Ge, Hang Qiu, et al. "WOMD-LiDAR: Raw Sensor Dataset Benchmark for Motion Forecasting". In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. 2024. DOI: 10.1109/ICRA57147.2024.10610651.

[48] M. Valverde, A. Moutinho, and J.-V. Zacchi. "A Survey of Deep Learning-Based 3D Object Detection Methods for Autonomous Driving Across Different Sensor Modalities". In: *Sensors* 25.17 (2025), p. 5264. DOI: 10.3390/s25175264.

[49] Nicolas Carion et al. "End-to-End Object Detection with Transformers". In: *European Conference on Computer Vision (ECCV)*. 2020, pp. 213–229.

[50] Xizhou Zhu et al. "Deformable DETR: Deformable Transformers for End-to-End Object Detection". In: *International Conference on Learning Representations (ICLR)*. 2021.

[51] Fangao Zeng et al. "MOTR: End-to-End Multiple-Object Tracking with Transformer". In: *European Conference on Computer Vision (ECCV)*. 2022, pp. 659–675.

[52] Xuyang Bai et al. "TransFusion: Robust LiDAR-Camera Fusion for 3D Object Detection with Transformers". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.

[53] Peng Gao et al. "Fast Convergence of DETR with Spatially Modulated Co-Attention". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 3621–3630.

[54] Matej Halinkovic, Alexey Vinel, and Wanda Benesova. "Li-ViP3D: Enhancing End-to-End Perception and Prediction with Camera-LiDAR Fusion". In: *2025 IEEE International Automated Vehicle Validation Conference (IAVVC)*. 2025, pp. 1–6.

[55] Matej Halinkovic et al. *Li-ViP3D++: Query-Gated Deformable Camera–LiDAR Fusion for End-to-End Perception and Trajectory Prediction*. 2026. arXiv: 2601.20720 [cs.CV].

[56] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[57] Xizhou Zhu et al. "Deformable ConvNets v2: More Deformable, Better Results". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 9308–9316.

[58] Tsung-Yi Lin et al. "Feature Pyramid Networks for Object Detection". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 2117–2125.

[59] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 5998–6008.

[60] Harold W. Kuhn. "The Hungarian Method for the Assignment Problem". In: *Naval Research Logistics Quarterly* 2.1–2 (1955), pp. 83–97.

[61] Charles R. Qi et al. "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 652–660.

[62] Hei Law and Jia Deng. "CornerNet: Detecting Objects as Paired Keypoints". In: *European Conference on Computer Vision (ECCV)*. 2018, pp. 734–750.

[63] Tsung-Yi Lin et al. “Focal Loss for Dense Object Detection”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2980–2988.

Appendix A

Description of Digital Submission

Thesis Evidence Number:
FIIT-XXXX-XXXXX

Obsah elektronického média je väčší ako 1GB a preto je odovzdaný vedúcemu záverečnej práce (doc. Ing. Giang Nguyen Thu, PhD.).

Name of submitted archive: `DP_MenoPriezvisko.zip`

The digital part included in the thesis contains the following files:

- Ubuntu OS: run in command line `$ tree` to get a directory structure as below.
- Inspiration from *https://github.com/FIIT-ISA/cookiecutter-data-science*

```
ROOT // Root directory.
│
├── flatbuffers // This flatbuffers director
└── scheme
```

```
├── LICENSE
├── Makefile           <- Makefile with commands like `make data` or `make train`
├── README.md          <- The top-level README for developers using this project.
├── data
│   ├── external       <- Data from third party sources.
│   ├── interim        <- Intermediate data that has been transformed.
│   ├── processed      <- The final, canonical data sets for modeling.
│   └── raw            <- The original, immutable data dump.
│
├── docs               <- A default Sphinx project; see sphinx-doc.org for details
│
├── models             <- Trained and serialized models, model predictions, or model summaries
│
├── notebooks          <- Jupyter notebooks. Naming convention is a number (for ordering),
│                         the creator's initials, and a short `-` delimited description, e.g.
│                         `1.0-jqp-initial-data-exploration`.
│
├── references         <- Data dictionaries, manuals, and all other explanatory materials.
│
├── reports            <- Generated analysis as HTML, PDF, LaTeX, etc.
│   └── figures        <- Generated graphics and figures to be used in reporting
│
├── requirements.txt   <- The requirements file for reproducing the analysis environment, e.g.
│                         generated with `pip freeze > requirements.txt`
│
├── setup.py           <- makes project pip installable (pip install -e .) so src can be impor
├── src                <- Source code for use in this project.
│   ├── __init__.py    <- Makes src a Python module
│   │
│   ├── data           <- Scripts to download or generate data
│   │   └── make_dataset.py
│   │
│   ├── features       <- Scripts to turn raw data into features for modeling
│   │   └── build_features.py
│   │
│   ├── models         <- Scripts to train models and then use trained models to make
│   │   │                 predictions
│   │   ├── predict_model.py
│   │   └── train_model.py
│   │
│   └── visualization  <- Scripts to create exploratory and results oriented visualizations
```

Figure A.1: The structure of the ZIP file

Appendix B

Technical Documentation

B.1 Installation Manual

1. Clone the project from the Git repository (URL) OR unzip the ZIP file ...
2. Run *pip install -r requirements.txt* in the root folder of the project
3. Run *python -m spacy download en* to have the data corpus ...

B.2 User Manual / Používateľská príručka

1. Spustenie programu funguje pomocou hlavného Python súboru *main.py* príkazom *python main.py*.
2. Po spustení programu sa zobrazí grafické rozhranie programu v ktorom sa dá hneď na hlavnej obrazovke spustiť sumarizácia pre jeden vstupný súbor.
3. Tlačidlami v hornej časti obrazovky sa dá presúvať medzi funkcionalitami programu ako je viac súborová sumarizácia alebo zobrazovanie článkov. Na týchto obrazovkách sa tlačidlom *Back to main menu* dá vrátiť späť na hlavnú obrazovku.

Screenshot(s) používateľského grafického rozhrania (GUI) ako obrázok (Figure)

(Another variant without GUI:) The main objective of this work is to ... Several case studies were presented that examine the capabilities of ... to show the feasibility and achievability of ... Users in our work are considered to work directly with the code as described in Section B.1.

B.3 Developer Manual

if needed then add ...

Appendix C

Work Schedule

C.1 Work Schedule for BP1/DP1

Week	Plan
1. - 2.	...
3. - 4.	...
5. - 6.	...
7. - 8.	...
9. - 10.	...
11. - 12.	...

Some explanation text... (if needed) ... a few sentences

Self-Evaluation of Work Schedule for BP1/DP1: Vyjadrenie k plneniu plánu: some text ... one paragraph ... self-evaluation

(In the best case:) The BP/DP 1 work plan is considered satisfied and successful.

C.2 Work Schedule for BP2/DP2

Week	Plan
1. - 2.	...
3. - 4.	...
5. - 6.	...
7. - 8.	...
9. - 10.	...
11. - 12.	...

Some explanation text... (if needed) ... a few sentences

Self-Evaluation of Work Schedule for BP2/DP2: Vyjadrenie k plneniu plánu: some text ... one paragraph ... self-evaluation

(In the best case:) The BP/DP 2 work plan is considered satisfied and successful.

C.3 Work Schedule for DP3

Week	Plan
1. - 2.	...
3. - 4.	...
5. - 6.	...
7. - 8.	...
9. - 10.	...
11. - 12.	...

Some explanation text... (if needed) ... a few sentences

Self-Evaluation of Work Schedule for DP3: Vyjadrenie k plneniu plánu: some text ... one paragraph ... self-evaluation

(In the best case:) The DP 3 work plan is considered satisfied and successful.